

Maschinelles Lernen

Prof. Dr. Christine Preisach



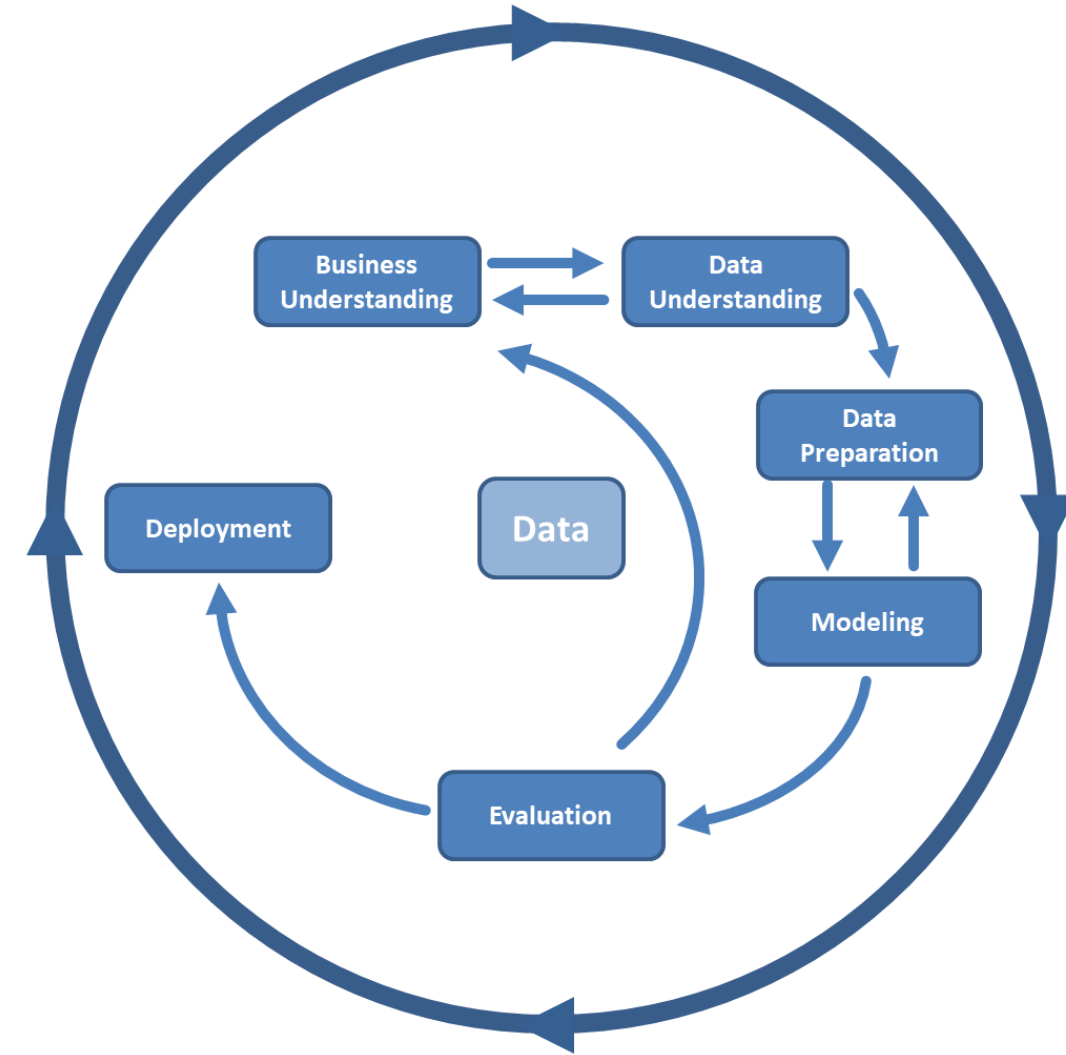
Transformationen



Überblick Transformationen

Data Preparation: Transformationen

- Hauptkomponentenanalyse (PCA)
- Kernel PCA



Motivation: Dimensionsreduktion

Unter der **Dimensionsreduktion** verstehen wir die Verringerung der Dimension durch Auswahl vorhandener oder Bildung neuer Merkmale.

Eine **Reduktion der Dimension** des Datenbestands hat mitunter erhebliche **Vorteile**:

- Rauschen und irrelevante Merkmale können reduziert werden
- Leichtere Visualisierung
- Geringere Rechenzeiten

Allgemein haben viele Mustererkennungsverfahren **Schwierigkeiten mit hohen Dimensionen**:

- Sparsity: Die Daten besetzen den Merkmalsraum (mit zunehmender Dimension) immer **dünnere**.
- Abstände werden weniger bedeutend.

kNN ist viel empfindlicher bzgl. hoher Dimensionen als SVM oder gar Neuronale Netze

Manuelle Merkmalsauswahl

Durch (manuelle oder heuristische) **Auswahl geeigneter Merkmale** kann man die **Dimension der Merkmale reduzieren**.

Man unterscheidet dabei zwischen

- irrelevanten und
- redundanten

Merkmalen.

Irrelevante Merkmale sind solche Merkmale, die (fast) keine nützliche Information enthalten.

Beispiel: Immatrikulationsnummer haben keinen Bezug zu den Leistungen der Studierenden.

Redundante Merkmale sind solche Merkmale, die bereits in anderen Merkmalen enthaltene Information »verdoppeln«.

Beispiel: Preis eines Produkts und die dafür gezahlte Mehrwertsteuer.

Automatische Merkmalsauswahl

Qualitativ idealer Ansatz: Brute-Force – Wende die Mustererkennung auf alle möglichen Teilmengen von Merkmalen an und entscheide nach dem besten Ergebnis (z.B. `fselect` von `libsvm`)

Aber: 2^d mögliche Kombinationen!

Grundsätzlich existieren folgende Standardansätze zur Merkmalsauswahl

- Eingebettete Ansätze
- Filteransätze und
- Wrapper-Ansätze
- Transformationen

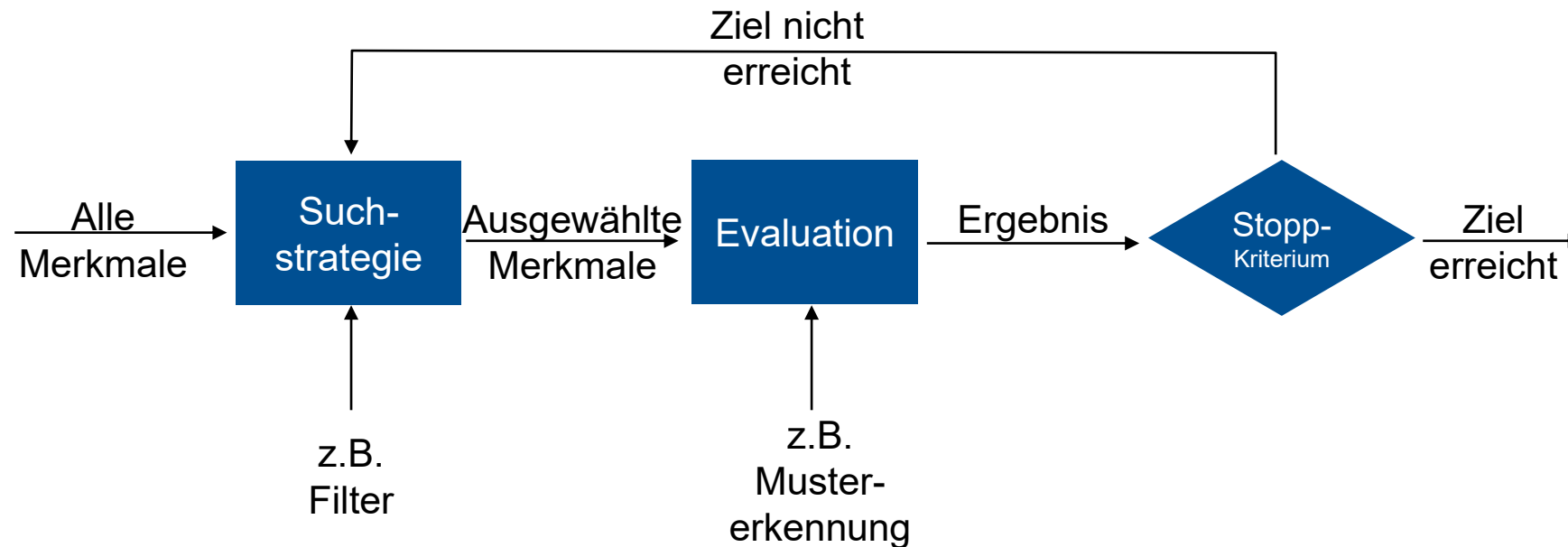
Bei **eingebetteten Ansätzen** werden die Merkmale **implizit** während der Mustererkennung ausgewählt (so z.B. `featureSelect` bei Entscheidungsbäumen insb. `random forests`).

Automatische Merkmalsauswahl

Bei **Filteransätzen** werden die Merkmale vor der Mustererkennung und unabhängig von dem dort eingesetzten Verfahren ausgewählt (z.B. via statistischen Tests oder Korrelationsanalyse).

Wrapper-Ansätze gleichen dem idealen Ansatz, adressieren aber **nicht alle möglichen Kombinationen** von Merkmalen (z.B. forward selection).

In Kombination liefern beide Ansätze z.B. folgende Architektur



Automatische Merkmalsauswahl

Qualitativ idealer Ansatz: Brute-Force – Wende die Mustererkennung auf alle möglichen Teilmengen von Merkmalen an und entscheide nach dem besten Ergebnis (z.B. `fselect` von `libsvm`)

Aber: 2^d mögliche Kombinationen!

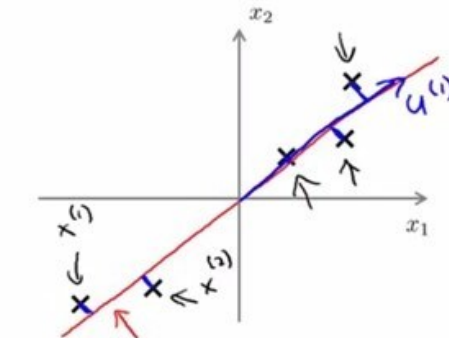
Grundsätzlich existieren folgende Standardansätze zur Merkmalsauswahl

- Eingebettete Ansätze
- Filteransätze und
- Wrapper-Ansätze
- **Transformationen**

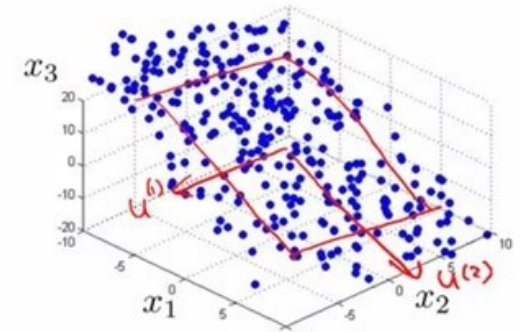
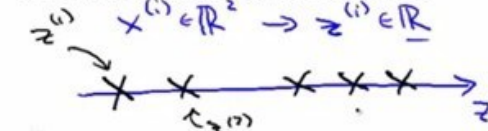
Dimensionsreduktionsverfahren - PCA - Principal Component Analysis

- Bei der Dimensionsreduktion werden Daten so in neue Dimensionen transformiert, dass einige Dimensionen verworfen werden können, ohne dass wichtige Informationen verloren gehen
- **Ziel der PCA (Hauptkomponentenanalyse):** Aus den ursprünglichen (oft korrelierten) Variablen, wenige **unkorrelierte Variablen** (Komponenten) extrahieren. Komponenten werden nach 'Wichtigkeit' (Anteil an erklärter Varianz) geordnet.
- **Die Hauptkomponenten sind neue Features**, die als **Linearkombinationen der ursprünglichen Variablen** erzeugt werden. Die Hauptkomponenten sind **unkorreliert**, und die meisten Informationen in den ursprünglichen Variablen werden in den **ersten Komponenten** gespeichert.
- Die Varianz in der niedrigdimensionalen Darstellung der Daten soll maximiert werden.
- Die enthaltenen Informationen werden anhand der Kovarianz der Daten gemessen.

Principal Component Analysis (PCA) algorithm



Reduce data from 2D to 1D



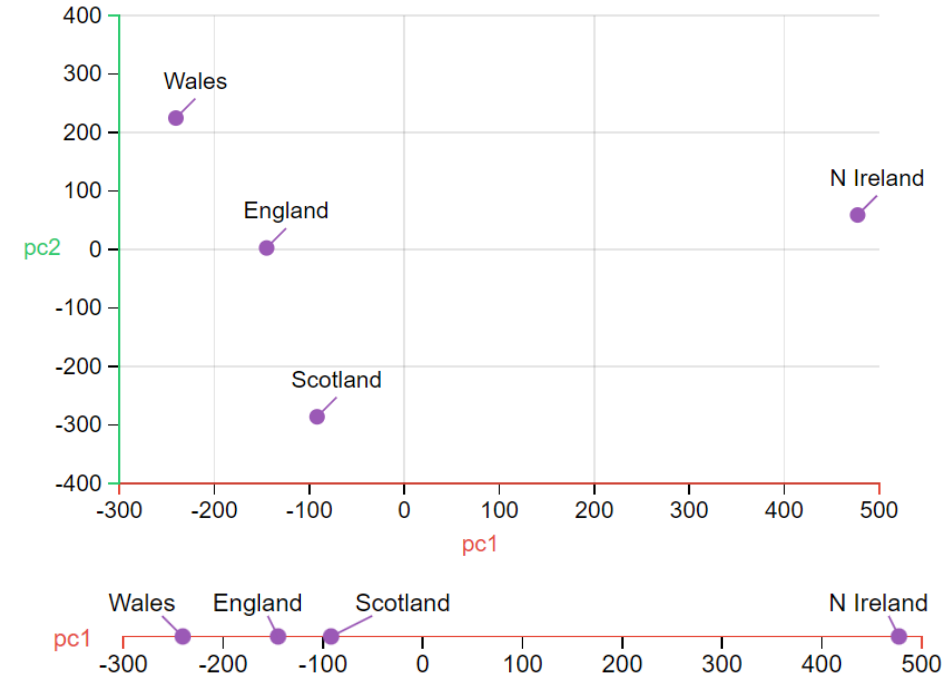
Reduce data from 3D to 2D

PCA - Beispiel

	England	N Ireland	Scotland	Wales
Alcoholic drinks	375	135	458	475
Beverages	57	47	53	73
Carcase meat	245	267	242	227
Cereals	1472	1494	1462	1582
Cheese	105	66	103	103
Confectionery	54	41	62	64
Fats and oils	193	209	184	235
Fish	147	93	122	160
Fresh fruit	1102	674	957	1137
Fresh potatoes	720	1033	566	874
Fresh Veg	253	143	171	265
Other meat	685	586	750	803
Other Veg	488	355	418	570
Processed potatoes	198	187	220	203
Processed Veg	360	334	337	365
Soft drinks	1374	1506	1572	1256
Sugars	156	139	147	175

- Datensatz mit 17 Dimensionen zum Ess- und Trinkverhalten verschiedener Länder in Großbritannien.

<https://setosa.io/ev/principal-component-analysis/>



- PCA findet ein **neues Koordinatensystem**, in dem jeder Punkt einen neuen (x,y) Wert hat, so dass die Varianz maximal ist.
- PCA ist hilfreich, um Dimensionen zu eliminieren, die nicht nützlich sind.
- PC2 trägt weniger zur Varianz des Datensatzes bei.

PCA – Zentrierung

Gegeben sei eine Menge von N Objekten, deren **Merkmale** alle in vektorieller Darstellung $\vec{g}_1, \dots, \vec{g}_N$ vorliegen

$$\vec{g}_n = (x_{1,n}, \dots, x_{d,n})^T, n = 1, \dots, N$$

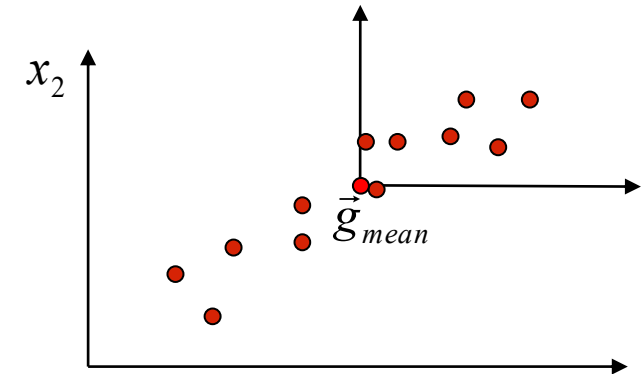
Wie bei der Mahalanobis-Distanz berechnen wir die **Kovarianzmatrix** der Daten, d.h. zunächst werden die Vektoren bezüglich ihres Mittelwerts

$$\vec{g}_{mean} = \frac{1}{N} \sum_{n=1}^N \vec{g}_n$$

für $n = 1, \dots, N$ zentriert:

$$\Delta \vec{g}_n = \vec{g}_n - \vec{g}_{mean}$$

Erinnerung: Das entspricht einer **Verschiebung der Daten in ein neues Koordinatensystem:**



PCA - Principal Component Analysis - Hauptkomponentenanalyse

Die N zentrierten Vektoren werden spaltenweise in die Datenmatrix

$$G = (\Delta \vec{g}_1 | \dots | \Delta \vec{g}_N) \in \mathbb{R}^{d \times N}$$

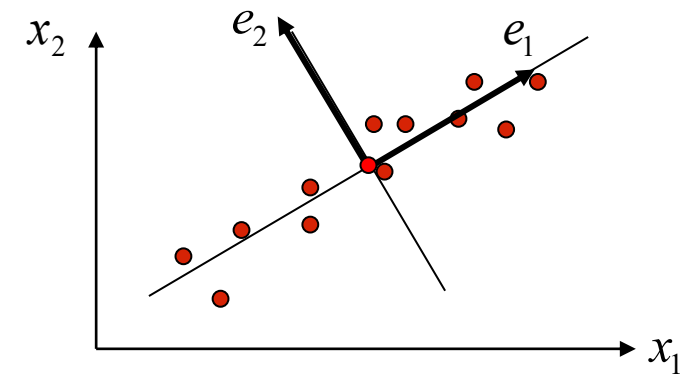
geschrieben und daraus die (empirische) **Kovarianzmatrix** berechnet $C = \frac{1}{N-1} G G^T \in \mathbb{R}^{d \times d}$

Das **gesuchte Koordinatensystem** mit der

- **größten Streuung** längs der ersten Achse
- **zweitgrößten Streuung** längs der zweiten Achse
- ...
- kleinsten Streuung längs der d -ten Achse

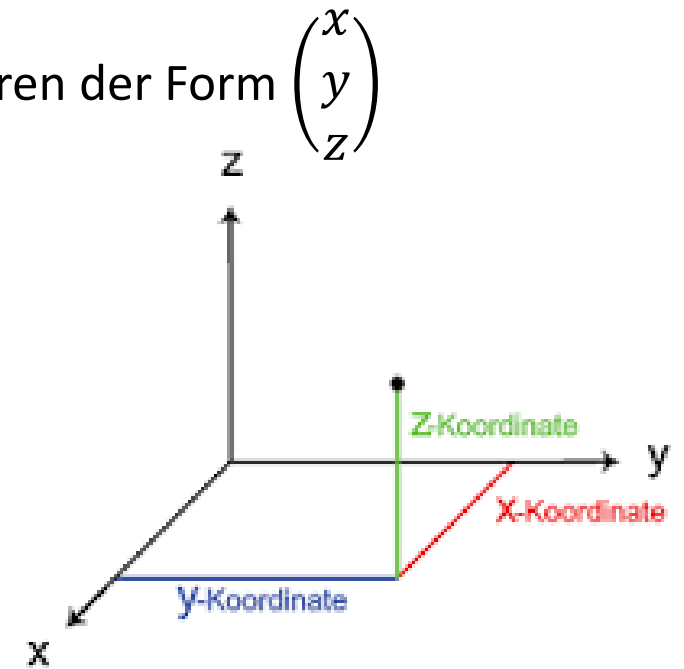
ist gegeben durch die **Eigenvektoren der Kovarianzmatrix C**

PCA (Principal Component Analysis) verwendet die **Eigenvektoren der Kovarianzmatrix**, um die Daten in ein **neues Koordinatensystem** zu verschieben bzw. das Koordinatensystem zu drehen.



Wiederholung: Vektoren und Vektorraum

- Ein **Vektorraum** oder linearer Raum ist eine algebraische Struktur, die in vielen Teilgebieten der Mathematik verwendet wird.
- Die Elemente eines Vektorraums heißen **Vektoren**. Sie können addiert oder mit Skalaren (reelle Zahlen) multipliziert werden, das Ergebnis ist wieder ein Vektor desselben Vektorraums.
- **Beispiele:**
 - das **kartesische Koordinatensystem** ist ein Vektorraum, er entspricht dem $\mathbb{R}^2$ und ist 2-dimensional und wird **euklidische Ebene** genannt
 - Der 3-dimensionale Raum $\mathbb{R}^3$ ist auch ein Vektorraum und enthält Vektoren der Form $\begin{pmatrix} x \\ y \\ z \end{pmatrix}$
 - Die Gesamtheit **aller n-dimensionalen Vektoren** (Vektoren mit n Komponenten) wird **n-dimensionaler Vektorraum** genannt



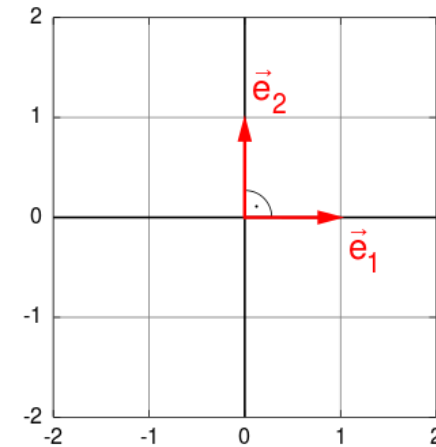
Wiederholung: Vektoren und Vektorraum

- **Eigenschaften:**

- Vektoren sind die Elemente des Vektorraums
- Ein Vektorraum erlaubt es Vektoren über Rechenoperationen - Addition, Subtraktion und Multiplikation zu verknüpfen
- **Basis eines Vektorraums:** besteht aus Vektoren mit denen alle anderen Vektoren „gebaut“ werden können

- Die Standardbasis im $\mathbb{R}^2$ sind das $\vec{e}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$, $\vec{e}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$

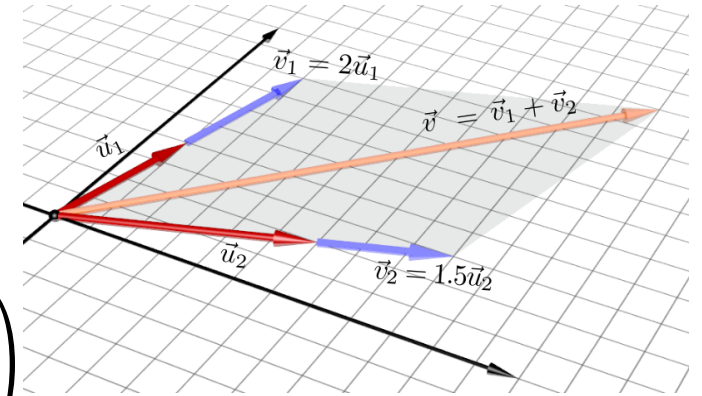
- Die Standardbasis im $\mathbb{R}^3$ sind das $\vec{e}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$, $\vec{e}_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$, $\vec{e}_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$



- **Dimension** d des Vektorraums entspricht der Anzahl der Basisvektoren (z.B. d=2, oder d=3)

Wiederholung: Vektoren und Vektorraum

- Alle Vektoren lassen sich aus Linearkombinationen (Vektoraddition und skalare Multiplikation) von Basisvektoren erstellen:
- $\vec{d} = \alpha_1 e_1 + \dots + \alpha_n e_n = \sum_{i=1}^n \alpha_i e_i$
- **Beispiel:** $\vec{d} = \begin{pmatrix} 5 \\ -1 \\ 3 \end{pmatrix} = 5 * e_1 - 1 * e_2 + 3 * e_3 = 5 * \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} - \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} + 3 * \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$
- Nicht nur Standardbasisvektoren sondern auch andere Basisvektoren, die **linear unabhängig** voneinander sind, lassen sich nutzen, um Vektoren über eine Linearkombination darzustellen.

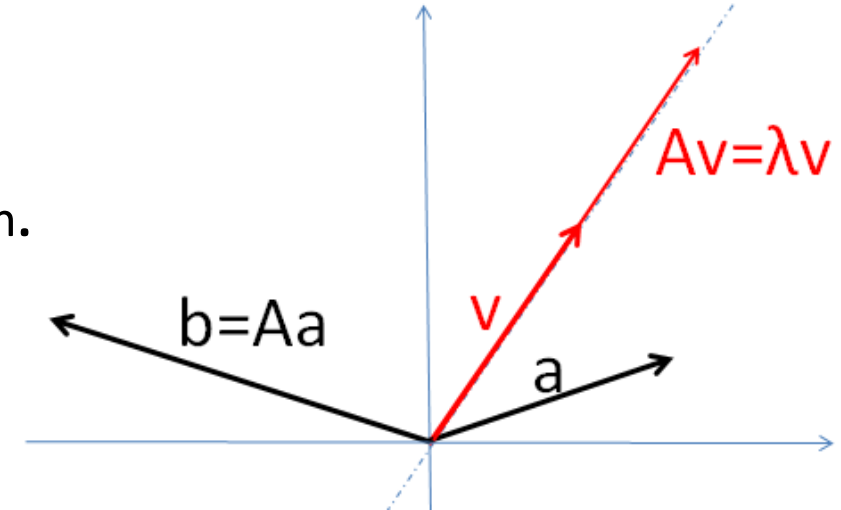


Wiederholung: Eigenvektoren

- **Eigenvektoren** sind spezielle Vektoren, die bei der Anwendung einer **linearen Transformation** (dargestellt als Matrix) ihre **Richtung beibehalten**, jedoch in ihrer **Länge skaliert** werden (aber nicht gedreht oder gekippt werden). Das bedeutet, dass wenn man eine Matrix auf einen Eigenvektor anwendet, **das Ergebnis ein Vielfaches des ursprünglichen Eigenvektors** ist.
- **Eigenwerte sind die Skalierungsfaktoren**, die angeben, um wie viel der Eigenvektor gestreckt oder gestaucht wird. Wenn (A) eine Matrix ist, $(\vec{v})$ ein Eigenvektor und λ der zugehörige Eigenwert, dann gilt die Beziehung:

$$A\vec{v} = \lambda\vec{v}$$

- Eigenvektoren helfen uns, komplexe Transformationen zu vereinfachen.
- **Eigenvektoren** können eine **Basis für einen Vektorraum** bilden, wenn sie auf 1 normiert werden.



<http://elsenaju.info/Determinanten/Eigenwerte.htm>

Wozu Eigenvektoren?

- **Google's PageRank-Algorithmus:** Eigenvektoren helfen, die Wichtigkeit von Webseiten zu berechnen.
- **Hauptkomponentenanalyse (PCA):** In der Datenanalyse werden Eigenvektoren genutzt, um die wichtigsten Muster in großen Datensätzen zu finden.
- **Schwingungen:** In der Physik beschreiben Eigenvektoren, wie sich Objekte in bestimmten Richtungen bewegen (z. B. bei Brücken oder Gebäuden).
- **Computer Vision:** Eigenvektoren werden genutzt, um Gesichter oder Objekte in Bildern zu erkennen.

<http://elsenaju.info/Determinanten/Eigenwerte.htm>

Wiederholung: Eigenvektoren

Definition: Ein (ggf. komplexer) Vektor $\vec{e}$ ist **Eigenvektor einer Matrix C** , wenn es eine (ggf. komplexe) Zahl λ gibt, sodass gilt: $C\vec{e} = \lambda\vec{e}$

Eigenschaften von Eigenvektoren:

A. Es ist

$$C\vec{e} = \lambda\vec{e} \Leftrightarrow C\vec{e} - \lambda\vec{e} = \vec{0} \Leftrightarrow (C - \lambda I)\vec{e} = \vec{0}$$

Zu einem **gegebenen Eigenwert λ** liefert also das **Gleichungssystem**

$$(C - \lambda I)\vec{e} = \vec{0}$$

einen **zugehörigen Eigenvektor $\vec{e}$** .

B. Ist $\vec{e}$ Eigenvektor einer Matrix C mit Eigenwert λ , so gilt für jede Zahl α

$$C(\alpha\vec{e}) = \alpha(C\vec{e}) = \alpha(\lambda\vec{e}) = \lambda(\alpha\vec{e})$$

insbesondere ist also $\alpha\vec{e}$ auch ein Eigenvektor zum Eigenwert.

Wiederholung: Berechnung von Eigenvektoren

Die **Lösung des linearen Gleichungssystems** gilt:

Insbesondere sind nur solche λ 's Eigenwerte, für die das Gleichungssystem

$$(C - \lambda I)\vec{e} = \vec{0}$$

keine eindeutige Lösung besitzt, also für die insb. gilt

$$\det(C - \lambda I) = 0$$

Eigenvektorberechnung:

Schritt 1: Bestimme alle Eigenwerte $\lambda_1, \dots, \lambda_d$ mit

$$\det(C - \lambda_i I) = 0$$

Das ist ein
Polynom in der
Unbekannten λ !
Es heißt das
charakteristische
Polynom von C .

Schritt 2: Zu jedem dieser λ_i bestimme den zugehörigen **Eigenvektor** $\vec{e}_i$ durch Lösen des **Gleichungssystems**

$$(C - \lambda_i I)\vec{e}_i = \vec{0}$$

PCA – Beispiel

Zahlenbeispiel: Die Merkmalsvektoren seien gegeben

$$\vec{g}_1 = \begin{pmatrix} 3 \\ 3 \end{pmatrix}, \vec{g}_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \vec{g}_3 = \begin{pmatrix} 4 \\ 2 \end{pmatrix}, \vec{g}_4 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \vec{g}_5 = \begin{pmatrix} 5 \\ 5 \end{pmatrix}$$

Berechne $\vec{g}_{mean}$

Zentriere die Merkmalsvektoren

Und fasse zur Datenmatrix zusammen

Kovarianzmatrix berechnen $C = \frac{1}{N-1} G G^T \in \mathbb{R}^{d \times d}$

PCA – Beispiel

Zahlenbeispiel: Die Merkmalsvektoren seien gegeben

$$\vec{g}_1 = \begin{pmatrix} 3 \\ 3 \end{pmatrix}, \vec{g}_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \vec{g}_3 = \begin{pmatrix} 4 \\ 2 \end{pmatrix}, \vec{g}_4 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \vec{g}_5 = \begin{pmatrix} 5 \\ 5 \end{pmatrix}$$

Berechne $\vec{g}_{mean}$

$$\vec{g}_{mean} = \frac{1}{N} \sum_{n=1}^N \vec{g}_n = \frac{1}{5} \begin{pmatrix} 3 + 1 + 4 + 2 + 5 \\ 3 + 1 + 2 + 4 + 5 \end{pmatrix} = \frac{1}{5} \begin{pmatrix} 15 \\ 15 \end{pmatrix} = \begin{pmatrix} 3 \\ 3 \end{pmatrix}$$

Zentriere die Merkmalsvektoren

$$\Delta\vec{g}_1 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \Delta\vec{g}_2 = \begin{pmatrix} -2 \\ -2 \end{pmatrix}, \Delta\vec{g}_3 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \Delta\vec{g}_4 = \begin{pmatrix} -1 \\ 1 \end{pmatrix}, \Delta\vec{g}_5 = \begin{pmatrix} 2 \\ 2 \end{pmatrix}$$

Und fasse zur Datenmatrix zusammen:

$$G = \begin{pmatrix} 0 & -2 & 1 & -1 & 2 \\ 0 & -2 & -1 & 1 & 2 \end{pmatrix}$$

PCA – Beispiel

daraus die Kovarianzmatrix berechnet:

$$C = \frac{1}{4} \begin{pmatrix} 0 & -2 & 1 & -1 & 2 \\ 0 & -2 & -1 & 1 & 2 \end{pmatrix} \begin{pmatrix} 0 & 0 \\ -2 & -2 \\ 1 & -1 \\ -1 & 1 \\ 2 & 2 \end{pmatrix} = \frac{1}{4} \begin{pmatrix} 10 & 6 \\ 6 & 10 \end{pmatrix}$$

Berechne die Eigenwerte und Eigenvektoren

Wiederholung: Berechnung von Eigenvektoren

Die **Lösung des linearen Gleichungssystems** gilt:

Insbesondere sind nur solche λ 's Eigenwerte, für die das Gleichungssystem

$$(C - \lambda I)\vec{e} = \vec{0}$$

keine eindeutige Lösung besitzt, also für die insb. gilt

$$\det(C - \lambda I) = 0$$

Eigenvektorberechnung:

Schritt 1: Bestimme alle Eigenwerte $\lambda_1, \dots, \lambda_d$ mit

$$\det(C - \lambda_i I) = 0$$

Schritt 2: Zu jedem dieser λ_i bestimme den zugehörigen **Eigenvektor** $\vec{e}_i$ durch Lösen des **Gleichungssystems**

$$(C - \lambda_i I)\vec{e}_i = \vec{0}$$

Schritt 1: Charakteristisches Polynom

Das charakteristische Polynom wird aufgestellt

$$\det(C - \lambda I) = \det\left(\frac{1}{4}\begin{pmatrix} 10 & 6 \\ 6 & 10 \end{pmatrix} - \lambda\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}\right) = \det\begin{pmatrix} \frac{10}{4} - \lambda & \frac{6}{4} \\ \frac{6}{4} & \frac{10}{4} - \lambda \end{pmatrix}$$

$$= \left(\frac{10}{4} - \lambda\right)^2 - \left(\frac{6}{4}\right)^2 = \frac{100}{16} - 2\frac{10}{4}\lambda + \lambda^2 - \frac{36}{16}$$

$$= \lambda^2 - \underbrace{5}_{p:=}\lambda + \underbrace{\frac{100-36}{16}}_{q:=} = \lambda^2 - \underbrace{5}_{p:=}\lambda + \underbrace{\frac{16}{4}}_{q:=}$$

Es werden die
Indizes der
Eigenwerte
immer
absteigend
 $\lambda_1 \geq \dots \geq \lambda_d$
vergeben!

$$\lambda_{1/2} = -\frac{p}{2} \pm \sqrt{\left(\frac{p}{2}\right)^2 - q} = \frac{5}{2} \pm \sqrt{\frac{25}{4} - \frac{16}{4}} = \frac{5}{2} \pm \sqrt{\frac{9}{4}} = \frac{5}{2} \pm \frac{3}{2}$$

und dessen Nullstellen bestimmt:

$$\lambda_1 = \frac{5}{2} + \frac{3}{2} = 4$$

$$\lambda_2 = \frac{5}{2} - \frac{3}{2} = 1$$

Lineare Gleichungssysteme - Matrix Schreibweise

- Ein **lineares Gleichungssystem** (kurz LGS) ist in der linearen Algebra **eine Menge linearer Gleichungen** mit einer oder mehreren Unbekannten, die alle gleichzeitig erfüllt sein sollen.
- Lineare Gleichungssysteme werden oft in **Matrixschreibweise** dargestellt.
- Sie bestehen aus einer **Koeffizientenmatrix A**, dem Vektor x der die Variablen enthält (und dem Lösungsvektor entspricht) und dem Vektor b mit der die **rechte Seite** repräsentiert.
- Ein LGS wird durch eine **Matrix-Vektor-Multiplikation** dargestellt

$$\begin{aligned}a_{11}x_1 + a_{12}x_2 + a_{13}x_3 &= b_1, \\a_{21}x_1 + a_{22}x_2 + a_{23}x_3 &= b_2, \\a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3.\end{aligned}$$

$$A = \begin{pmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix} \quad x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}; \quad b = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}$$

- Man gibt den Vektor x normalerweise nicht an:

$$\left(A \mid b \right) = \left(\begin{array}{cccc|c} a_{1,1} & a_{1,2} & \cdots & a_{1n} & b_1 \\ a_{2,1} & a_{2,2} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \\ a_{m1} & a_{m2} & \cdots & a_{mn} & b_m \end{array} \right)$$

Schritt 2: Auflösen der Gleichungssysteme

Berechnung der Eigenvektoren durch Lösung der Gleichung $(C - \lambda_1 I)\vec{e}_1 = \vec{0}$

$$(C - \lambda_1 I) = \frac{1}{4} \begin{pmatrix} 10 & 6 \\ 6 & 10 \end{pmatrix} - 4 \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} \frac{10}{4} - 4 & \frac{6}{4} \\ \frac{6}{4} & \frac{10}{4} - 4 \end{pmatrix} = \frac{1}{4} \begin{pmatrix} -6 & 6 \\ 6 & -6 \end{pmatrix}$$

Elementare Zeilenumformungen ergeben:

$$\frac{1}{4} \begin{pmatrix} -6 & 6 & | & 0 \\ 6 & -6 & | & 0 \end{pmatrix} \rightarrow \begin{pmatrix} -1 & 1 & | & 0 \\ 1 & -1 & | & 0 \end{pmatrix} \rightarrow \begin{pmatrix} -1 & 1 & | & 0 \\ 0 & 0 & | & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & -1 & | & 0 \\ 0 & 0 & | & 0 \end{pmatrix}$$

Setze also $\vec{e}_1$ so dass gilt

$$\begin{pmatrix} 1 & -1 \\ 0 & 0 \end{pmatrix} \vec{e}_1 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

also z.B. $\vec{e}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$

Schritt 2: Auflösen der Gleichungssysteme

Analog ergibt sich aus $(C - \lambda_2 I)\vec{e}_2 = \vec{0}$

$$\begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} \vec{e}_2 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

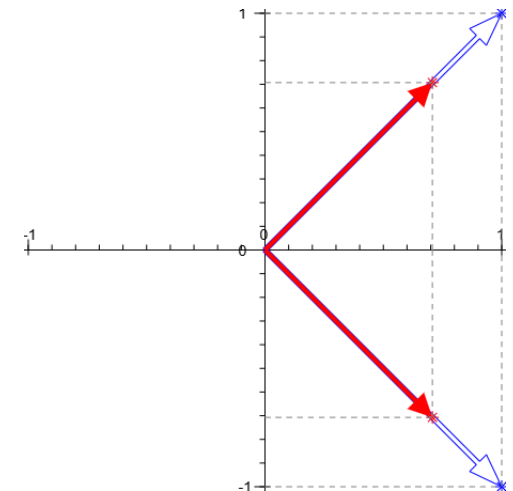
also z.B. die Lösung $\vec{e}_2 = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$

$$\text{Probe: } (C - \lambda I)\vec{e}_1 = \frac{1}{4} \begin{pmatrix} -6 & 6 \\ 6 & -6 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

$$(C - \lambda I)\vec{e}_2 = \frac{1}{4} \begin{pmatrix} -6 & 6 \\ 6 & -6 \end{pmatrix} \begin{pmatrix} -1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

Eigenschaften von Eigenvektoren

- Für jeden **Eigenwert** gibt es **unendlich viele Lösungen** des Gleichungssystems $(C - \lambda I)\vec{e} = \vec{0}$
- Der Lösungsraum von $(C - \lambda I)\vec{e} = \vec{0}$ heißt Eigenraum von C zum Eigenwert λ .
- Somit gibt es auch **unendlich viele Eigenvektoren, d.h. Eigenvektoren sind nicht eindeutig bestimmt**
- Wenn $\vec{e}$ ein Eigenvektor ist dann ist es auch $\alpha\vec{e}$
- Aber diese Lösungen sind nicht völlig unstrukturiert, sondern sie liegen alle auf der derselben Geraden/Ebene.
- Man nennt diese Gerade/Ebene einen (linearen) **Unterraum**. Und es ist möglich, zwei unabhängige Vektoren zu finden, welche dann diesen Unterraum aufspannen. Diese beiden Vektoren bilden dann also eine **Basis** dieses Unterraumes. Es gibt aber mehrere Möglichkeiten, eine solche Basis anzugeben. **Eigenvektoren sind eine solche Basis.**
- Wenn man die Eigenvektoren aber auf 1 normiert dann sind sie eindeutig.
- Die **maximale Anzahl an Eigenvektoren** einer Kovarianzmatrix mit Eigenwert ungleich 0 hängt von **der Anzahl der Instanzen und der Anzahl der Variablen** ab



Wiederholung: Norm bzw. Länge eines Vektors

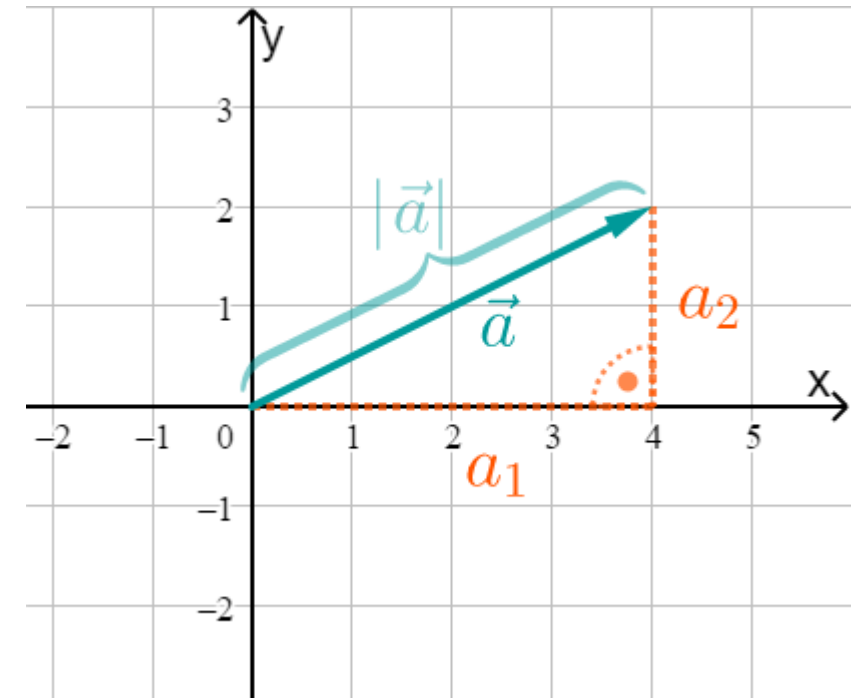
- **Die Länge eines Vektors oder der Betrag eines Vektors** kann man mit Hilfe des Satzes von Pythagoras berechnen:

- Für $\vec{a} = \begin{pmatrix} a_1 \\ a_2 \end{pmatrix}$

$$|\vec{a}| = \left| \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} \right| = \sqrt{a_1^2 + a_2^2}$$

- Satz von Pythagoras: $a^2 + b^2 = c^2$

Beispiel: $\vec{a} = \begin{pmatrix} 4 \\ 2 \end{pmatrix}$ $|\vec{a}| = \sqrt{16 + 4} = \sqrt{20} = 4,47$



Faktorisierung der Kovarianzmatrix

Die (normierten) Eigenvektoren und –werte können zu Matrizen zusammengefasst werden. Dabei werden die **Eigenvektoren** (zeigen die Richtung) **nach der Größe ihrer zugehörigen Eigenwerte sortiert**. Die Eigenwerte quantifizieren die **Varianz**, die jede Hauptkomponente erklärt.

$$U = (\vec{e}_1 | \dots | \vec{e}_d) \in \mathbb{R}^{d \times d} \text{ und } \Sigma = \begin{pmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \dots & 0 & \lambda_d \end{pmatrix} \in \mathbb{R}^{d \times d}$$

Gemeinsam bilden die Matrizen eine **Faktorisierung (Zerlegung) der Kovarianzmatrix**

$$C = U \Sigma U^T$$

Die Matrix U ist **orthogonal**, damit ist insbesondere $U^{-1} = U^T$ und sie beschreibt eine **Drehung/Spiegelung** im (zentrierten) Merkmalsraum.

Eigenschaften von Eigenvektoren

Die resultierenden Eigenvektoren $\vec{e}_i$ sind, absteigend sortiert nach dem Wert der zugehörigen Eigenwerte λ_i , die Hauptkomponenten der Ursprungsdaten. Die Eigenvektoren werden auf die Länge 1 normiert und in die Transformationsmatrix U eingetragen.

Beispiel:

$$\vec{e}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \lambda_1 = 4, \vec{e}_2 = \begin{pmatrix} -1 \\ 1 \end{pmatrix}, \lambda_2 = 1$$

$$|\vec{e}_1| = |\vec{e}_2| = \sqrt{2}$$

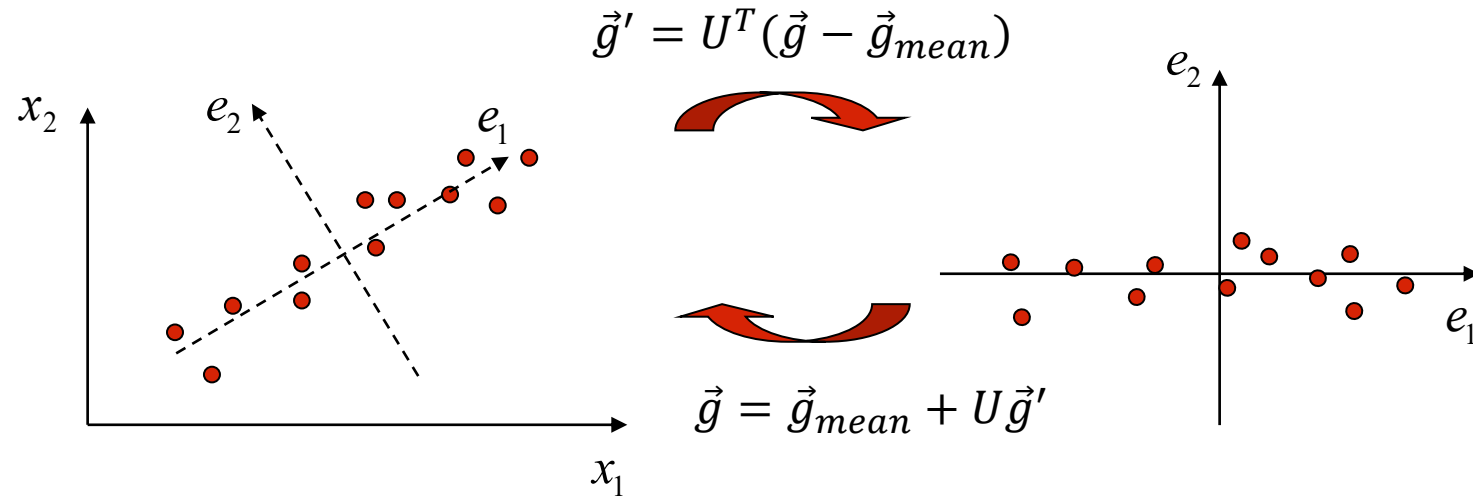
$$U = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$$

In diesem Beispiel haben die Vektoren die Länge $\sqrt{2}$, somit müssen die Vektoren mit $\frac{1}{\sqrt{2}}$ multipliziert werden.

Die Matrix U ist orthogonal, damit ist insbesondere $U^{-1} = U^T$ und sie beschreibt eine **Drehung** im (zentrierten) Merkmalsraum. Eine Matrix heißt orthogonal wenn das Produkt mit ihrer transponierten Matrix die Einheitsmatrix ergibt, bspw: $A^T \cdot A = I$. Durch Multiplikation mit einer orthogonalen Matrix können Vektoren gedreht oder gespiegelt werden.

Koordinatentransformation

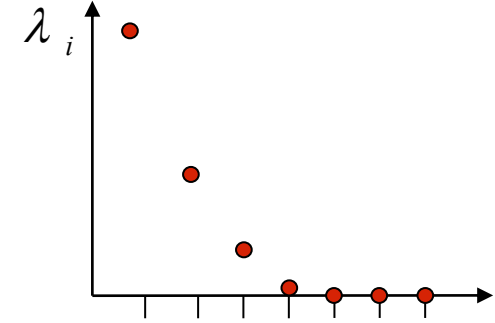
Mit Hilfe der Matrix U lassen sich Merkmalsvektoren aus dem ursprünglichen Koordinatensystem **in das neue Koordinatensystem mit der neuen Basis aus Eigenvektoren umrechnen** (transformieren):



Mit $\lambda_1 \geq \dots \geq \lambda_d \geq 0$ haben die Daten wie gewünscht längs der $\vec{e}_1$ -Achse die größte Streuung, längs der $\vec{e}_2$ -Achse die zweitgrößte Streuung usw.

Dimensionsreduktion

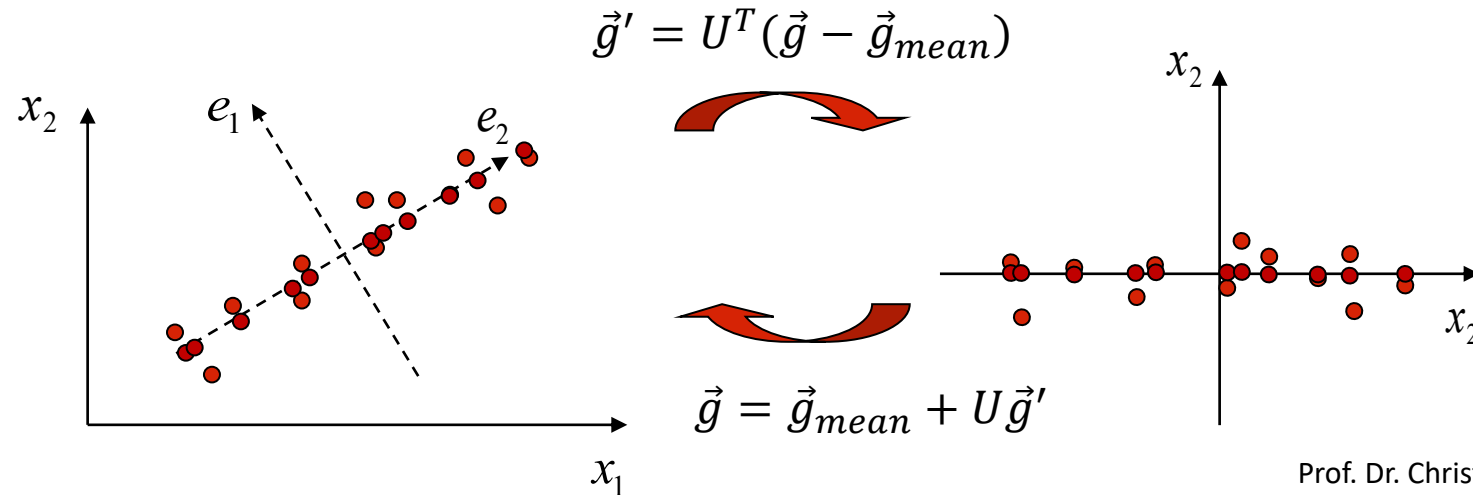
Bestehen zwischen den ursprünglichen Merkmalen **starke lineare Abhängigkeiten**, so haben die **Eigenwerte** einen schnell abfallenden Verlauf:



Die Eigenwerte quantifizieren die Varianz, die jede Hauptkomponente erklärt, und die Sortierung ermöglicht die Auswahl der informativsten Komponenten.

Für die **transformierten Merkmalsvektoren** bedeutet dies, dass wir alle **Komponenten** mit **kleinem** λ_i (=großem Index i) **weglassen** können ... **Dimensionsreduktion!**

z.B. alle Indizes i ,
für die gilt
 $\lambda_i < \frac{1}{10} \lambda_1$



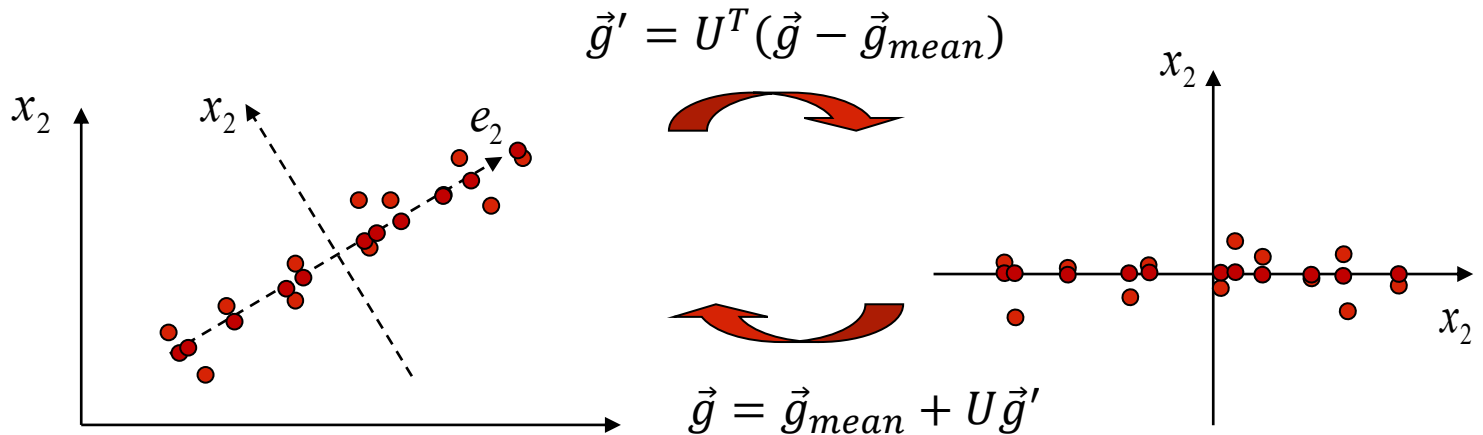
Zusammenfassung der Anwendung der PCA

Berechnung der PCA:

1. Berechne aus den Merkmalsvektoren $\vec{g}_1, \dots, \vec{g}_N$ von Objekten die Eigenwerte $\lambda_1 \geq \dots \geq \lambda_d \geq 0$ und die **Transformationsmatrix** (mit den Eigenvektoren)

$$U = (\vec{e}_1 | \dots | \vec{e}_d)$$

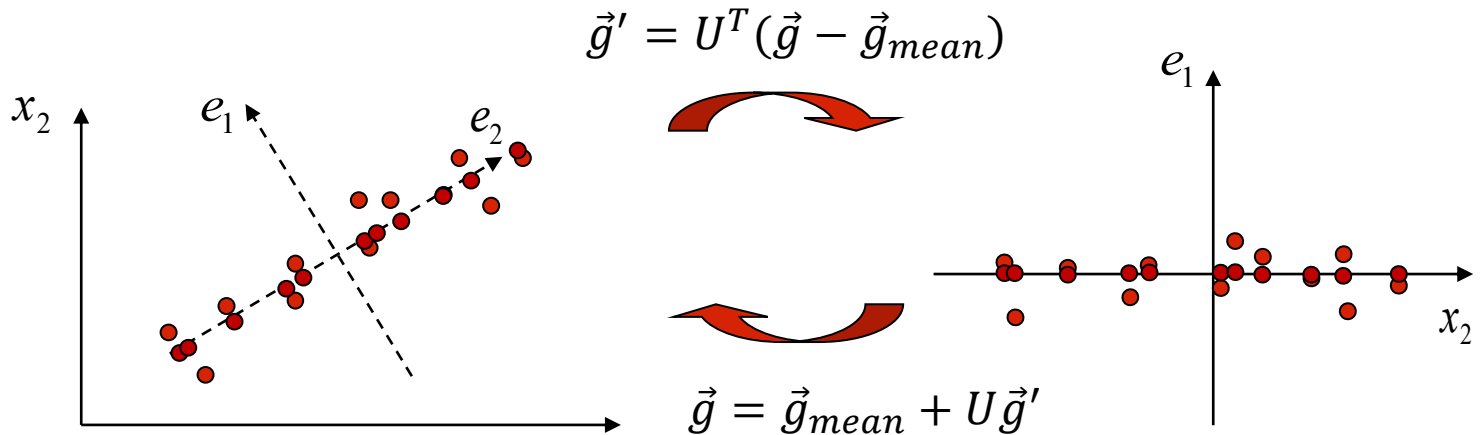
2. Lege die Anzahl Merkmale der neuen Merkmale i_0 fest.



Zusammenfassung der Anwendung der PCA

Anwendung der PCA:

1. Transformiere die Merkmalsvektoren mit $\vec{g}' = U^T(\vec{g} - \vec{g}_{mean})$ in den neuen Merkmalsraum.
2. Verwende nur noch die ersten i_0 Koordinaten der transponierten Vektoren.
3. Optional - Test: Transformiere den Vektor wieder zurück



Zahlenbeispiel

1. Transformiere einen Merkmalsvektor

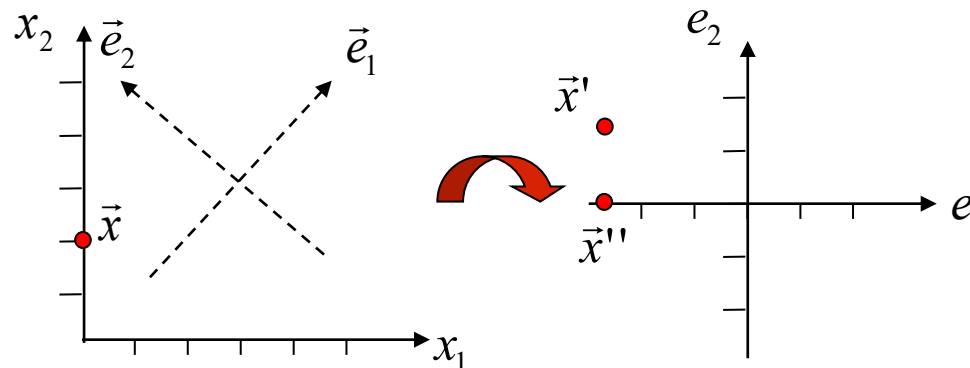
$$\vec{x} = \begin{pmatrix} 0 \\ 2 \end{pmatrix} \text{ mit } \vec{g}_{mean} = \begin{pmatrix} 3 \\ 3 \end{pmatrix}$$

$$\vec{x}' = U^T (\vec{x} - \vec{g}_{mean})$$

2. Setze alle Komponenten mit Index größer als $i_0 = 1$ auf 0

3. Test: Transformiere den Vektor wieder zurück

$$\vec{x} = \vec{g}_{mean} + U\vec{x}'$$



Zahlenbeispiel

1. Transformiere einen Merkmalsvektor mit

$$\vec{x} = \begin{pmatrix} 0 \\ 2 \end{pmatrix} \rightarrow \vec{x}' = \frac{1}{\sqrt{2}} \begin{pmatrix} -4 \\ 2 \end{pmatrix} \approx \begin{pmatrix} -2.83 \\ 1.41 \end{pmatrix}$$

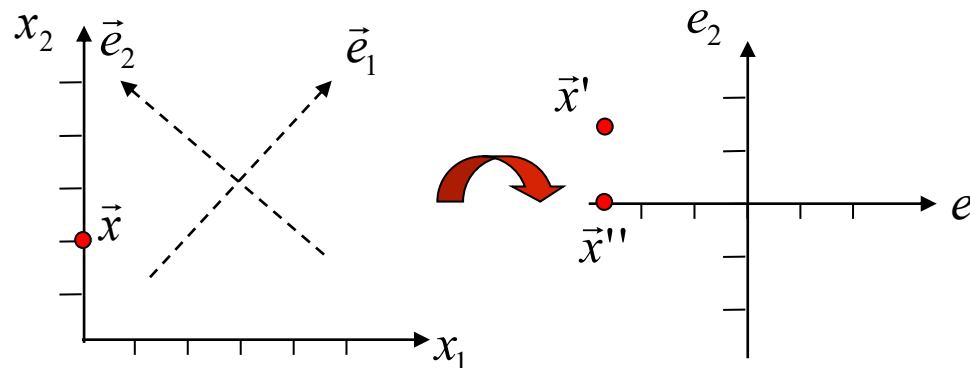
$$\vec{x}' = U^T (\vec{x} - \vec{g}_{mean})$$

2. Setze alle Komponenten mit Index größer als i_0 auf 0

$$\vec{x}' = \frac{1}{\sqrt{2}} \begin{pmatrix} -4 \\ 2 \end{pmatrix} \rightarrow \vec{x}'' = \frac{1}{\sqrt{2}} \begin{pmatrix} -4 \\ 0 \end{pmatrix}$$

3. Test: Transformiere den Vektor wieder zurück

$$\vec{x} = \vec{g}_{mean} + U\vec{x}'$$



Interpretation der PCA

Betrachten wir nochmals den Zusammenhang (**Rücktransformation**)

Affine Transformation

Translation Rotation

$$\vec{g} = \vec{g}_{mean} + U \vec{g}' = \vec{g}_{mean} + \sum_{i=1}^d x'_i \vec{e}_i$$

Vektor mit Koeffizienten im ursprünglich gegebenen Merkmalsraum

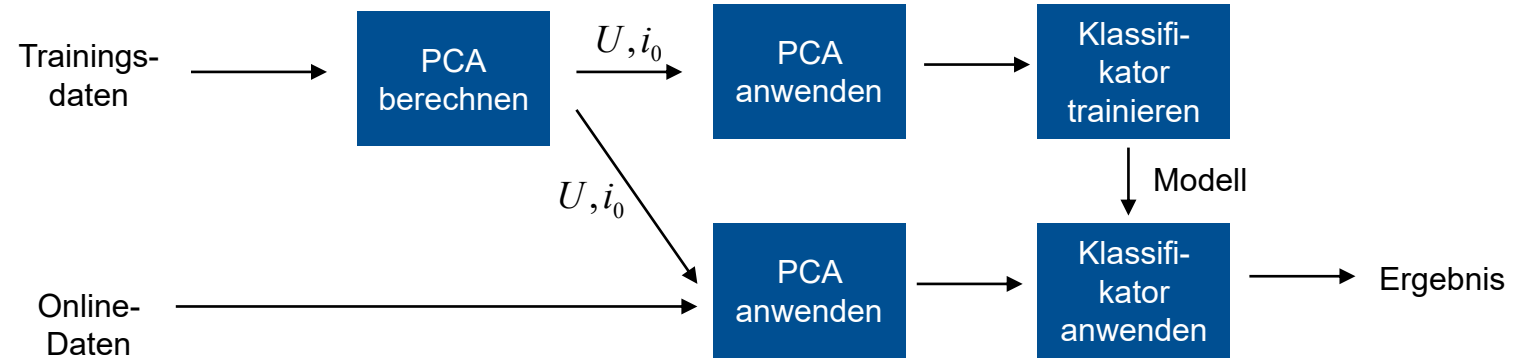
Linearkombination der Eigenvektoren, Basiswechsel

Koeffizienten des Vektors im neuen Merkmalsraum

Merkmalsvektoren lassen sich als **Linearkombination der Eigenvektoren** darstellen!

Beispiel PCA für Klassifikation

Für **Klassifikationsaufgaben** wird man die Transformation nur auf Basis der Trainingsmenge berechnen und dann mit den neuen – dimensionsreduzierten – Merkmalsvektoren einen Klassifikator trainieren.



Neue Merkmalsvektoren (Testdaten), also solche, die klassifiziert werden sollen, müssen dann auch transformiert und in der Dimension reduziert werden, bevor der Klassifikator angewandt werden kann.

PCA Coding

```
import numpy as np

class PCA:
    def __init__(self, n_components):
        self.n_components = n_components
        self.components = None
        self.mean = None

    def fit(self, X):
        # center the data
        self.mean = np.mean(X, axis=0)
        X = X - self.mean

        # compute the covariance matrix
        cov = np.cov(X, rowvar=False)

        # compute the eigenvalues and eigenvectors of the covariance matrix
        eigenvalues, eigenvectors = np.linalg.eigh(cov)

        # sort the eigenvalues and eigenvectors in decreasing order
        idx = np.argsort(eigenvalues)[::-1]
        eigenvalues = eigenvalues[idx]
        eigenvectors = eigenvectors[:, idx]

        # store the first n_components eigenvectors as the principal components
        self.components = eigenvectors[:, : self.n_components]

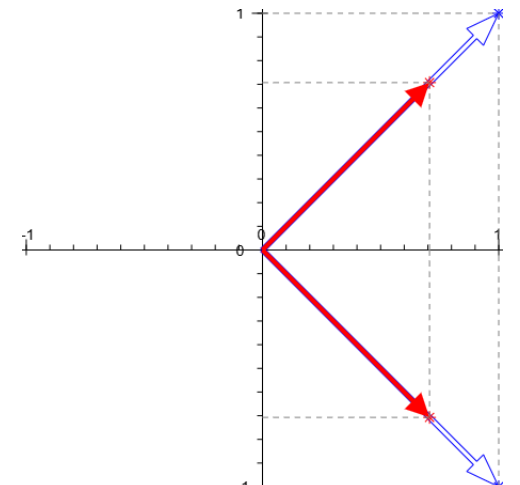
    def transform(self, X):
        # center the data
        X = X - self.mean

        # project the data onto the principal components
        X_transformed = np.dot(X, self.components)

        return X_transformed
```

Zusammenfassung

- Bei der Hauptkomponentenanalyse wird ein **Raum mit geringerer Dimensionalität** gesucht, der als Unterraum bezeichnet wird.
- Die **orthogonale Projektion** der Datenpunkte auf diesen Unterraum (über die Transformationsmatrix U , die die Eigenvektoren der Kovarianzmatrix enthält) soll die **Varianz** der projizierten Punkte **maximieren**.
- Die PCA sucht **orthogonale Richtungen maximaler Varianz** in zentrierten Daten. Diese **Richtungen** sind die **Eigenvektoren** der Kovarianzmatrix, die zugehörigen **Eigenwerte** sind die **Varianzen entlang dieser Richtungen**.
- Die neuen Merkmalsvektoren lassen sich als **Linearkombination der Eigenvektoren** darstellen.
- Die Hauptkomponenten sind **unkorreliert**, und die meisten Informationen in den ursprünglichen Variablen werden in den **ersten Komponenten** gespeichert.

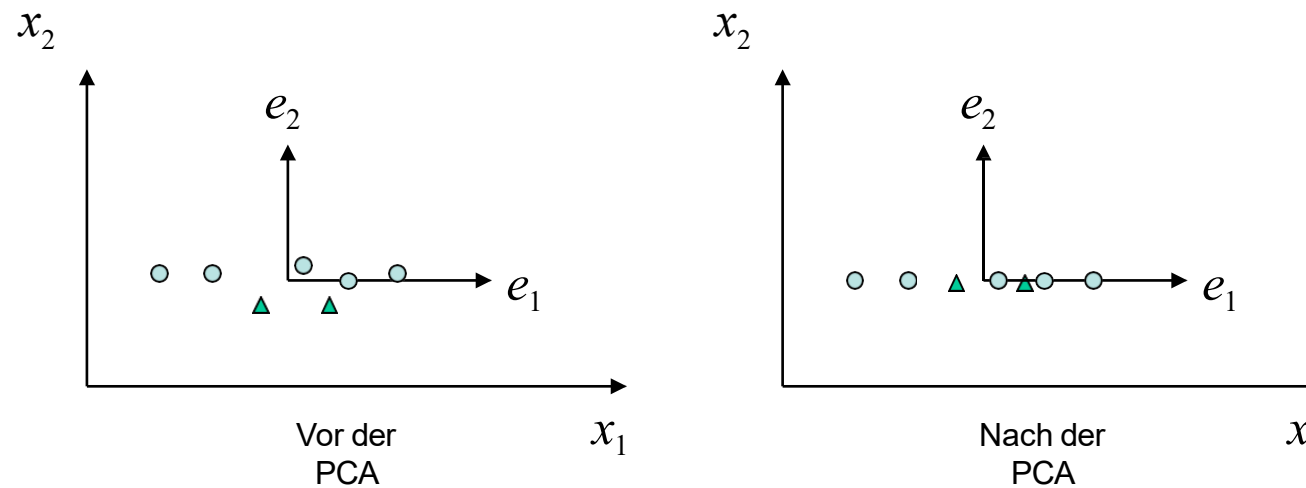


Vorteile der PCA

- **Dimensionsreduktion**
- Reduktion von Rauschen in den Daten
- Extraktion unkorrelierter Features die oft zu besseren Modellen führen
- Bessere Visualisierung
- Bessere Erkennung von Mustern

Zusammenfassung der Anwendung der PCA

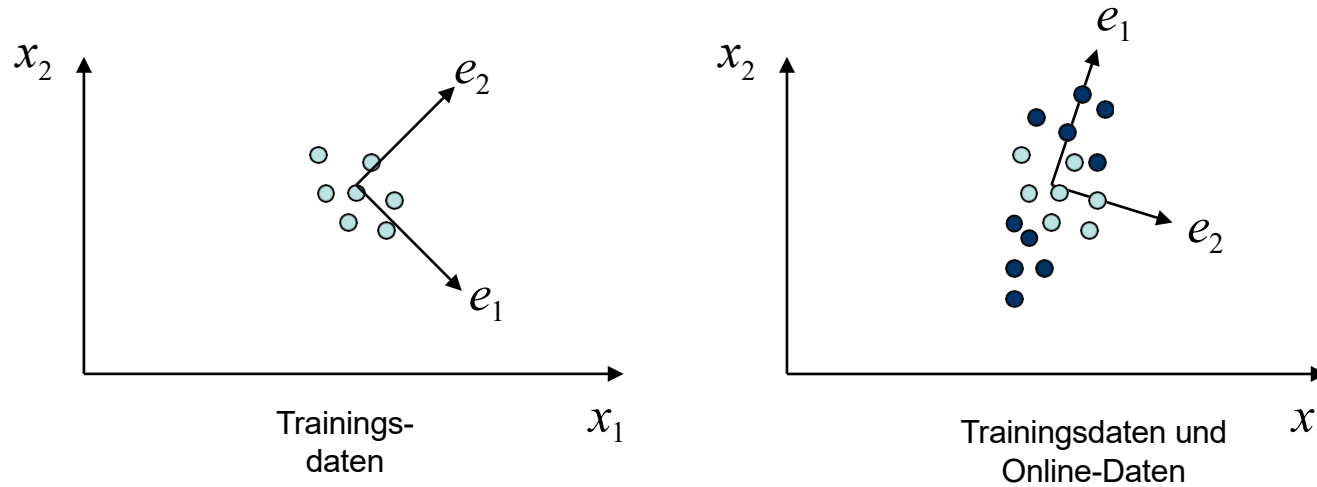
Achtung: Die PCA ist bei **unterschiedlichen Skalenniveaus** oder Einheiten nur mit Vorsicht einzusetzen...



Merkmale mit geringer Streuung werden »platt« gemacht. Auch dann, wenn sie die wirklich trennungswirksamen waren...

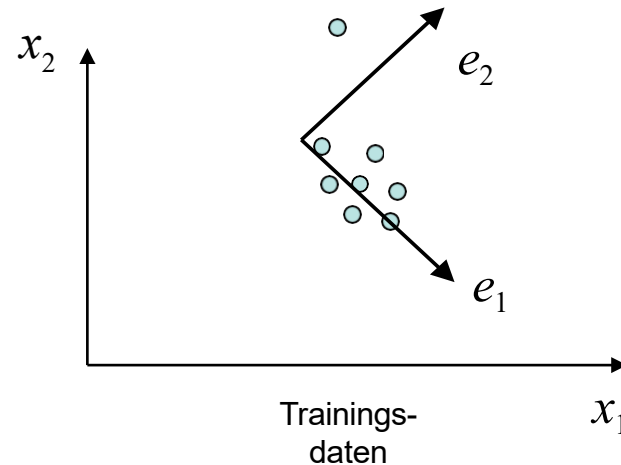
Zusammenfassung der Anwendung der PCA

Achtung: Auch für die PCA muss die Trainingsmenge repräsentativ sein:



Zusammenfassung der Anwendung der PCA

Achtung: Die PCA ist anfällig für **Ausreißer**. Ausreißer können die Berechnung der Hauptkomponenten erheblich beeinflussen, da sie die **Varianz** der Daten **verzerren** können. Dies kann dazu führen, dass die PCA eine **verzernte Darstellung** der zugrunde liegenden Struktur der Daten liefert.

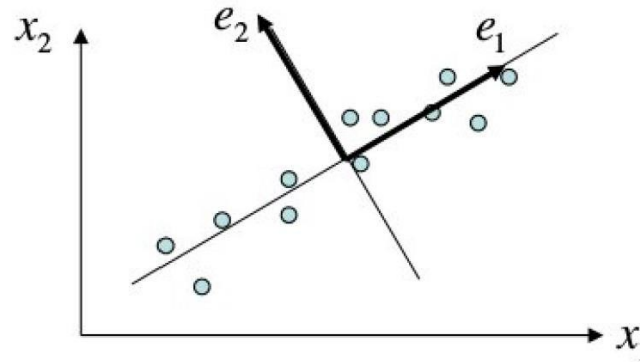
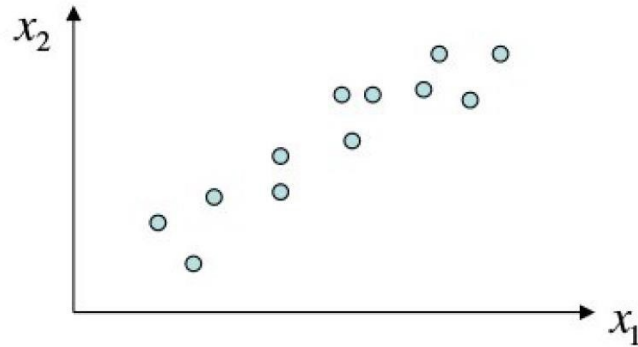


Nachteile der PCA

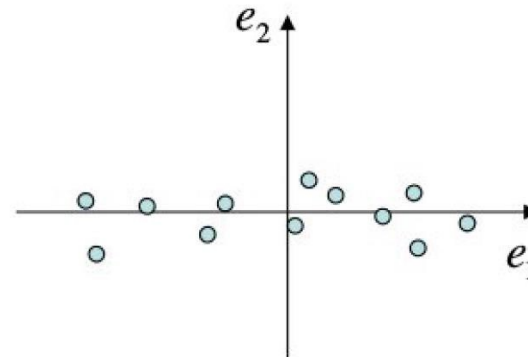
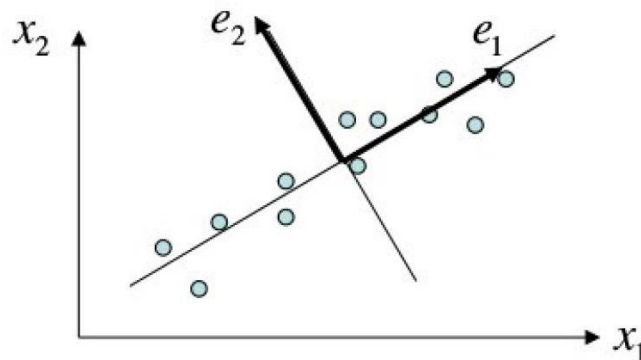
- Funktioniert nicht auf unterschiedlich skalierten Daten - Daten müssen normiert/standardisiert werden
- Ist für Daten mit Ausreißern nicht gut geeignet
- Die extrahierten Hauptkomponenten können inhaltlich **nicht gut interpretiert** werden
- PCA setzt voraus, dass die **Beziehungen** zwischen den Variablen **linear** sind

PCA - Auf einen Blick

1. Hauptkomponenten ermitteln

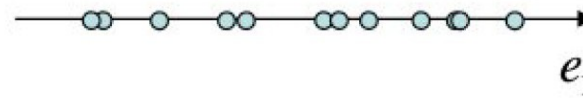
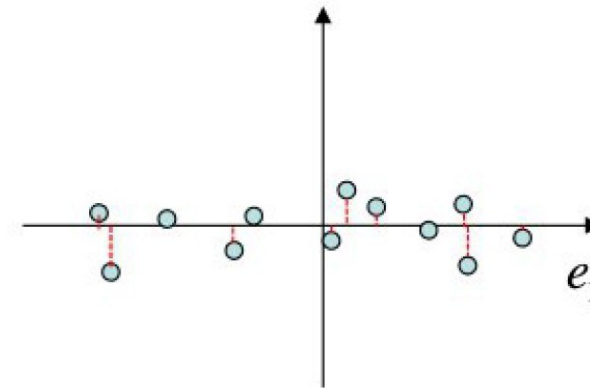
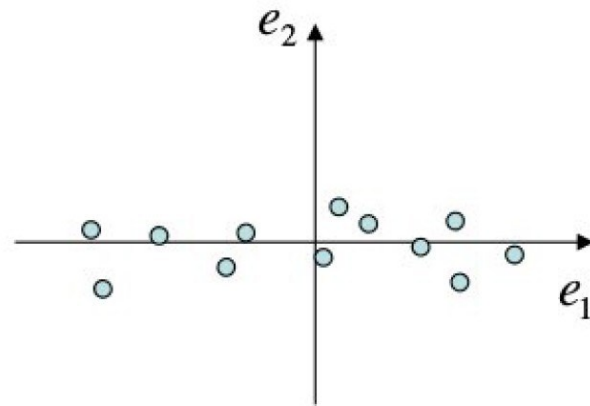


2. Koordinaten transformieren



PCA - Auf einen Blick

3. Dimensionsreduktion



Links

- [Anschauliche Beispiele der PCA](#)
- [PCA Tutorial mit Python](#)
- [Mathematics for Machine Learning](#) - PCA Kapitel 10 - S.317-347
- [Bishop-Pattern-Recognition-and-Machine-Learning](#) Kapitel 12.1

Anwendung - Bilderkennung

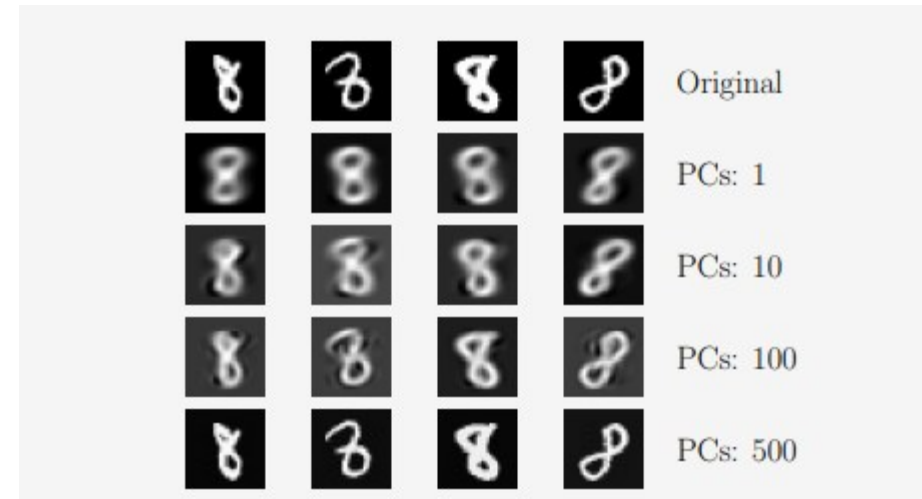


PCA in der Bildverarbeitung

- **Dimensionsreduktion:** PCA hilft, die Anzahl der Variablen (z. B. Pixel) zu reduzieren, ohne signifikante Informationen zu verlieren. Dies ist besonders nützlich bei **großen Bilddatensätzen**, da es die Berechnungszeit und den Speicherbedarf verringert.
- **Bildkompression:** Durch die Reduzierung der Dimensionen und das Speichern der Hauptkomponenten anstelle der ursprünglichen Bilddaten kann PCA zur **Kompression von Bildern** verwendet werden.
- **Rauschunterdrückung:** Durch die Beibehaltung nur der Hauptkomponenten, die den größten Teil der Varianz erklären, kann PCA helfen, Rauschen in Bildern zu reduzieren und die **Bildqualität zu verbessern**.
- **Merkmalerkennung:** PCA kann verwendet werden, um wichtige Merkmale in Bildern zu extrahieren, was für Aufgaben wie **Gesichtserkennung oder Objekterkennung** nützlich ist.
- **Bildklassifikation:** In Kombination mit anderen Algorithmen kann PCA dazu beitragen, die **Klassifikationsgenauigkeit** zu verbessern, indem es die Eingabedaten in einen Raum von Hauptkomponenten transformiert.

Beispiel: Handschriftenerkennung

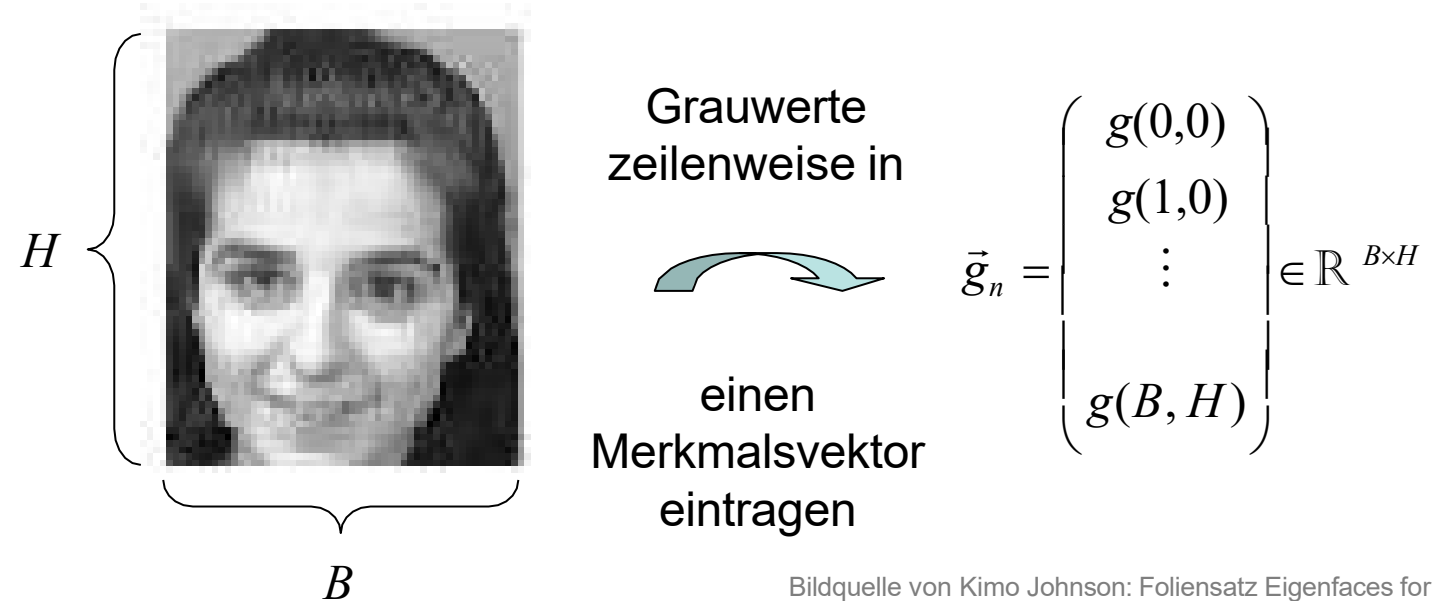
- MINST Datensatz - Handschriftliche Ziffern - $N = 60\,000$ Beispiele, jede Ziffer wird als Grauwertbild der Größe 28×28 gespeichert, besteht aus 784 Pixel
- Jedes Bild wird als Vektor der Dimension 784 dargestellt
- Wir verwenden 5.389 Trainingsbilder der Ziffer „8“ und bestimmen die Hauptkomponenten.
- Anschließend wurde die gelernte Projektionsmatrix U verwendet, um eine Reihe von Testbildern zu **rekonstruieren**
- Wir sehen, dass wir selbst mit einem **eindimensionalen Unterraum** eine halbwegs brauchbare Rekonstruktion der ursprünglichen Ziffern erhalten.
- Mit zunehmender Anzahl von Hauptkomponenten (PC) werden die Rekonstruktionen schärfer und mehr Details werden berücksichtigt.



Eigenfaces

In der Bildverarbeitung spricht man statt von PCA auch von der diskreten **Karhunen-Loève-Transformation** (KLT).

Bereits Anfang der 90er Jahre wurde die Hauptkomponentenanalyse (z.B. von Turk und Pentland) eingesetzt um Gesichter in Bildern zu klassifizieren.



Bildquelle von Kimo Johnson: Foliensatz Eigenfaces for Recognition

Eigenfaces - Daten

Diese Datenbasis hat $N=40$ Bilder mit jeweils $H=112$ und $B=92$, sodass jedes Bild die Abmessungen $H \times B$ hat. Somit besteht jedes Bild aus $d = 10.304$ Pixeln



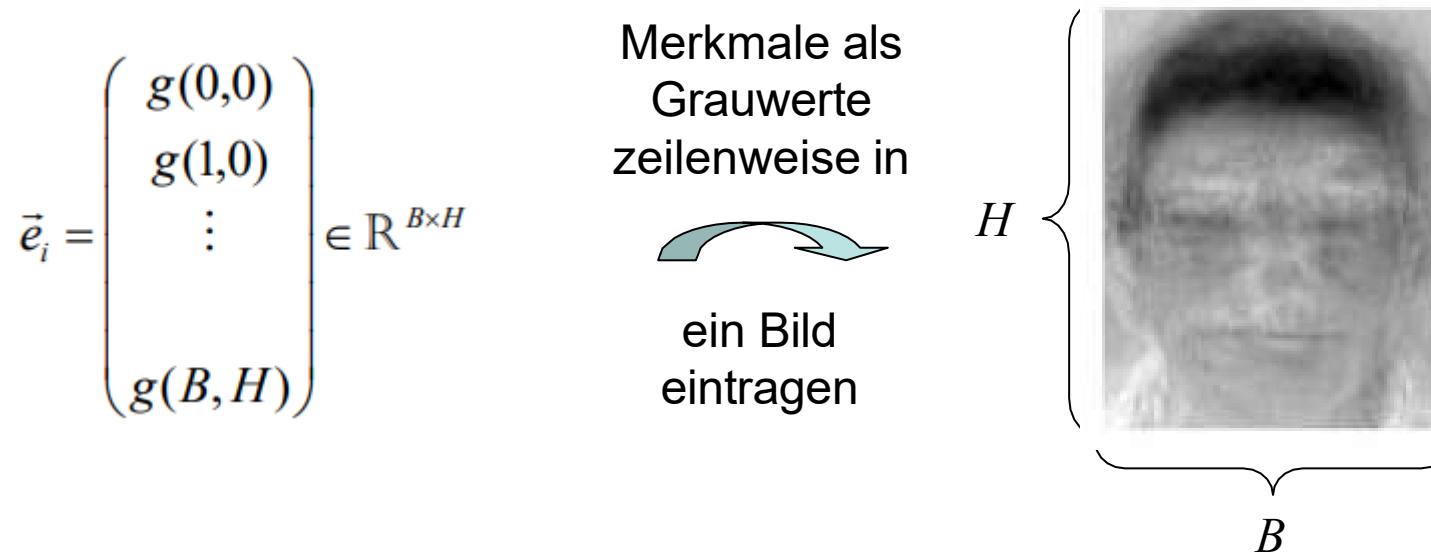
PCA - Eigenfaces - Vorgehen

1. Die einzelnen Bilder werden von der ursprünglichen Matrix zu einem **Vektor** umgeformt (hier: 10.304 Dimensionen).
2. Es wird das **Durchschnittsgesicht** aus dem gesamten Trainingsdatensatz berechnet.
3. Das Durchschnittsgesicht wird von jedem Bild aus dem Trainingsdatensatz abgezogen. Konkatenation der Differenzgesichter in einer **Datenmatrix**.
4. Berechnung der **Kovarianzmatrix**.
6. Berechnung der **Eigenvektoren** und **Eigenwerte**. Die Eigenvektoren besitzen dieselbe Dimensionalität wie die Originalbilder.
7. **Absteigende Sortierung der Eigenvektoren** nach der Größe der Eigenwerte. **Die ersten Eigenvektoren und Eigenwerte erfassen somit die meiste Varianz.**
8. Nutzung der PCA um **Gesichter kompakt darzustellen**, zu visualisieren, zu **rekonstruieren**, zu **klassifizieren**.
9. Wenn ein **neues Bild** eines Gesichtes analysiert werden soll, wird es in den Raum der Eigenvektoren projiziert, und die **Ähnlichkeit zu den bekannten Gesichtern wird gemessen**.

Visualisierung Eigenfaces

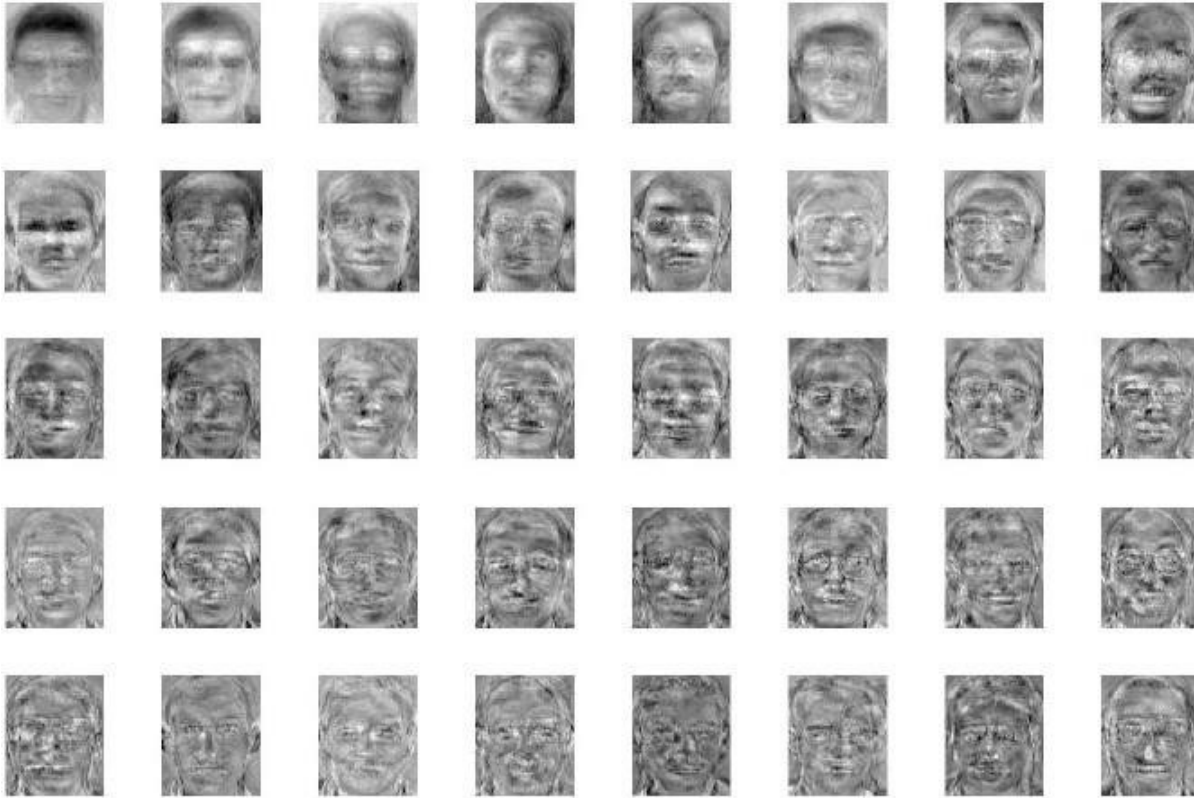
Dann kann die Matrix U mit den Basisvektoren $\vec{e}_1 \dots \vec{e}_d$ berechnet werden.

Die **Eigenvektoren** e_i heißen **Eigenfaces**. Man kann sie wieder in ein Bild eintragen (und dadurch visualisieren):



Anwendungen Eigenfaces

Wegen $N = 40 < d$ gibt es höchstens 40 Eigenwerte ungleich 0. Und das hier sind die zugehörigen Eigenvektoren:



Zu jedem (neuen) Bild $\vec{g}$ kann man dann mit

$$\vec{g}' = U^T(\vec{g} - \vec{g}_{mean})$$

dessen Koordinaten bzgl. der Eigenfaces berechnen

und die ersten 40 Komponenten z.B. zum Klassifizieren nutzen.

Weitere Anwendungen

Rekonstruktion: Die Trainingsbilder können wegen

$$\vec{g} = \vec{g}_{mean} + \sum_{i=1}^d x'_i \vec{e}_i$$

aus den Eigenfaces exakt rekonstruiert werden.

Synthese: Aus Eigenfaces können nach demselben Prinzip neue Bilder erzeugt werden:

Bei diesem
Beispiel wurde
 $\vec{g}_{mean}$ bereits auf
die Eigenvektoren
aufaddiert...



Kompression: Bilder können durch ihre Hauptkomponenten approximiert werden.

Eigenfaces Zusammenfassung

- Eigenfaces **reduzieren die Dimensionen** der Bilddaten erheblich, was die Berechnungszeit und den Speicherbedarf verringert. Dies erleichtert die Verarbeitung großer Bilddatensätze.
- Eigenfaces ermöglichen eine **effiziente und effektive Gesichtserkennung**, da nur die wichtigsten Merkmale der Gesichter analysiert und gleichzeitig die wichtigsten Informationen beibehalten werden.
- Eigenfaces sind **robust gegenüber kleinen Veränderungen** - sie können kleine Veränderungen in den Gesichtsausdrücke, Beleuchtung und Perspektive besser handhaben, da sie sich auf die Hauptmerkmale konzentrieren.
- Die Eigenfaces können als **Basisbilder** betrachtet werden, die die **Hauptmerkmale der Gesichter** darstellen, was eine interessante Möglichkeit zur Visualisierung von Gesichtsvariationen bietet.
- Dieses Verfahren wird in vielen Anwendungen eingesetzt zum Beispiel bei Sicherheitsüberwachungssystemen.

Achtung Falle

- Die PCA basiert auf einer numerischen Matrixoperation, der Diagonalisierung der Kovarianzmatrix
- $C = U\Sigma U^T$
- Wie bei allen numerischen Verfahren können **ungünstige Bedingungen** (z.B. Singularitäten durch dünne Besetzung der Matrix) zu Instabilitäten führen. Dies ist häufig der Fall, wenn die Matrix sehr groß ist!
- Beispiel Eigenfaces: $C = \frac{1}{N-1} G G^T \in \mathbb{R}^{d \times d}$
- In solchen Fällen geht man z.B. einen Weg über eine **Singulärwertzerlegung** der Datenmatrix G . Dazu später...
10.304 x 10.304 = 106.172.416!

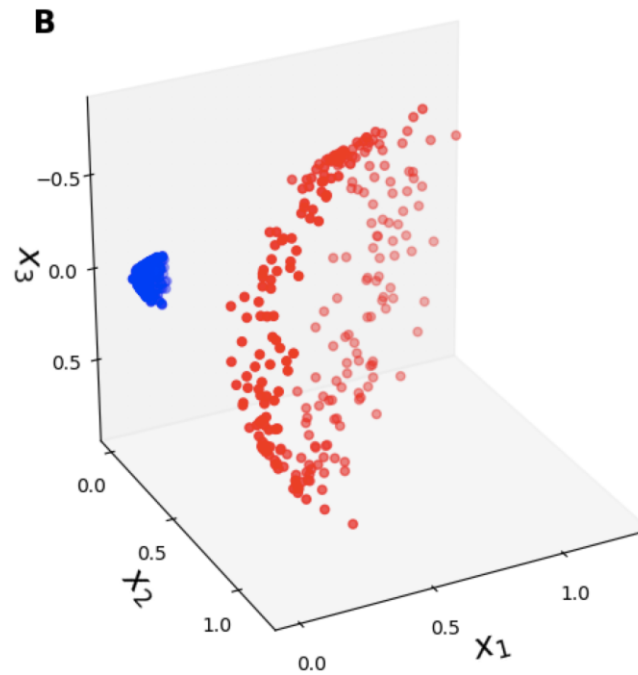
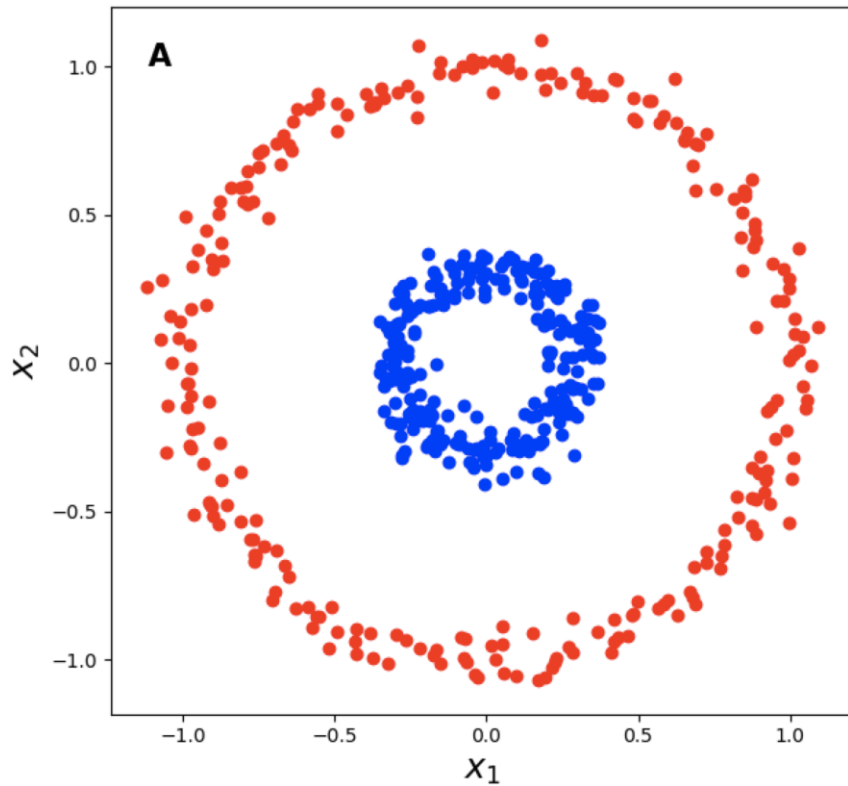
Nichtlineare PCA



Kernel PCA

Die Kernel PCA ist eine **Ergänzung der (linearen) PCA** für Anwendungsfälle, in denen die Daten **nichtlineare Zusammenhänge** besitzen.

Dafür müssen wir **Kernel** und den **Kernel Trick** kennenlernen.



Bildquelle <https://gregorygundersen.com/blog/2019/12/10/kernel-trick/>

Wiederholung Kovarianzmatrix & PCA

Die PCA basiert auf der **Faktorisierung der Kovarianzmatrix**, die wiederum lineare Zusammenhänge der Daten beschreibt.

Erinnerung: Die Kovarianzmatrix lässt sich aus den zentrierten Daten berechnen und ist für mehr als zwei, also z.B. d Merkmale $X_1, \dots, X_d$ definiert als:

$$\text{Cov}(X_i, X_j) = \frac{1}{N-1} G G^T \qquad G(X_i, X_j) = G G^T$$

Die Kovarianzmatrix entspricht einer normierten **Gram Matrix G**, sie enthält alle Skalarprodukte zwischen allen Paaren von Vektoren des Datensatzes. Die Kovarianzmatrix ist eine Skalarproduktmatrix der zentrierten Daten.

Die Idee der **Kernel PCA** ist nun die Kovarianzmatrix (genauer die Gramm-/Skalarproduktmatrix) durch eine **Kernel-Funktion** auf den Daten zu **ersetzen**

$$k(\vec{g}_i, \vec{g}_j) = \langle \phi(\vec{g}_i), \phi(\vec{g}_j) \rangle = \phi(\vec{g}_i)^T \phi(\vec{g}_j)$$

und damit **implizit** eine **nichtlineare Transformation** ϕ auf die Daten anzuwenden.

Matrizenmultiplikation

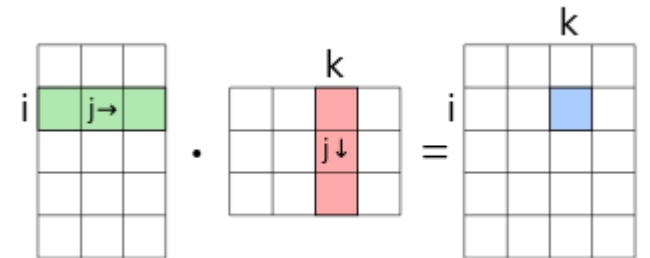
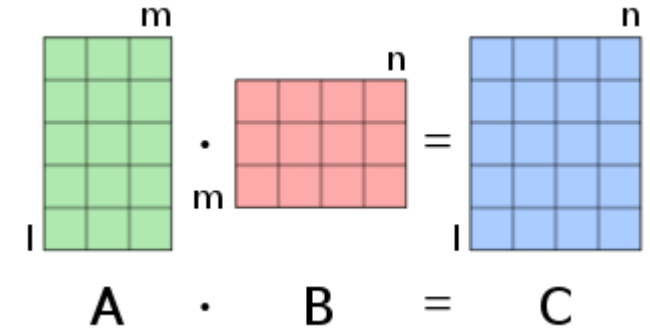
Zwei Matrizen können multipliziert werden, wenn die **Spaltenanzahl der linken mit der Zeilenanzahl der rechten Matrix übereinstimmt**. Das Produkt einer $l \times m$ Matrix mit einer $m \times n$ Matrix ergibt eine $l \times n$ Matrix

- Jeder Eintrag in der Ergebnis Matrix C wird aus der **komponentenweise Multiplikation** der Einträge der **i -ten Zeile von A mit der k -ten Spalte von B** und **Addition dieser Produkte** ermittelt

$$c_{ik} = \sum_{j=1}^m a_{ij} \cdot b_{jk}$$

- Bei zwei 3 x 3 Matrizen wird der erste Eintrag an Position c_{11} durch komponentenweise Multiplikation der ersten Zeile mit der ersten Spalte und der Addition dieser Produkte berechnet

$$c_{11} = a_{11} \cdot b_{11} + a_{12} \cdot b_{21} + a_{13} \cdot b_{31}$$

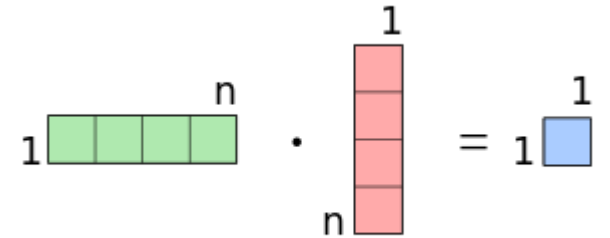


Matrizenmultiplikation - Spezialfälle

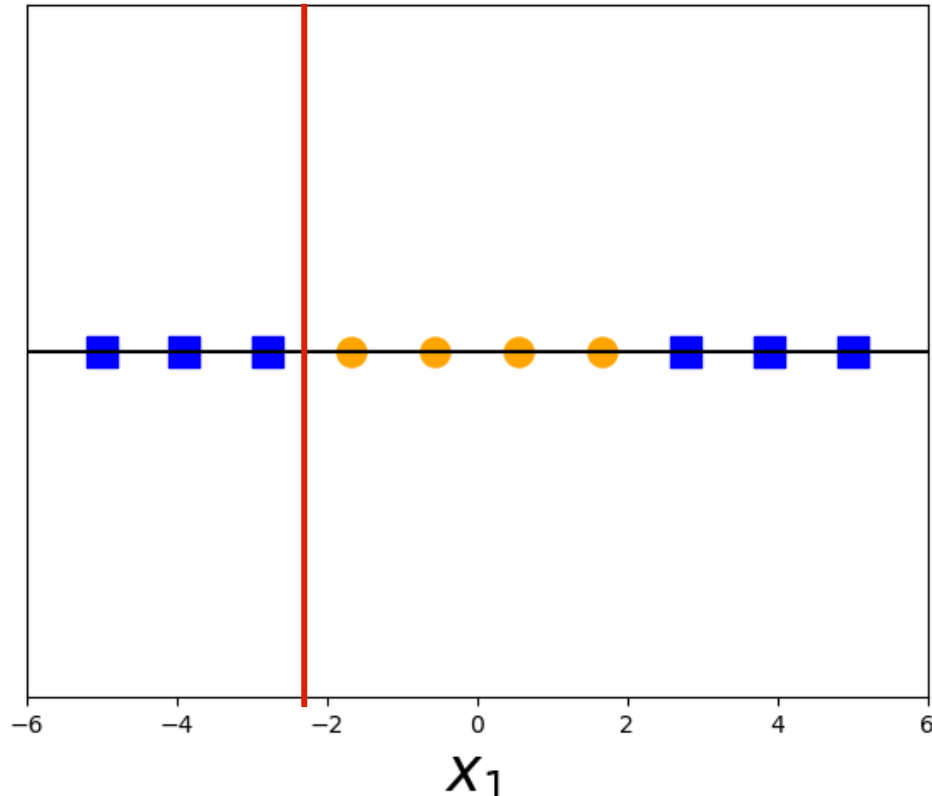
- Wenn wir einen Zeilenvektor also eine $1 \times m$ Matrix mit einem Spaltenvektor $m \times 1$ Matrix multiplizieren, entspricht das dem **Skalarprodukt**.

Vektornotation: $\vec{a} \cdot \vec{b} = a_1 b_1 + a_2 b_2 + a_3 b_3.$

Matrixnotation: $\langle x, y \rangle := x_1 y_1 + x_2 y_2 + \dots + x_n y_n = \sum_{i=1}^n x_i y_i = x^T y$



Nichtlinear separierbare Klassen – ein Beispiel



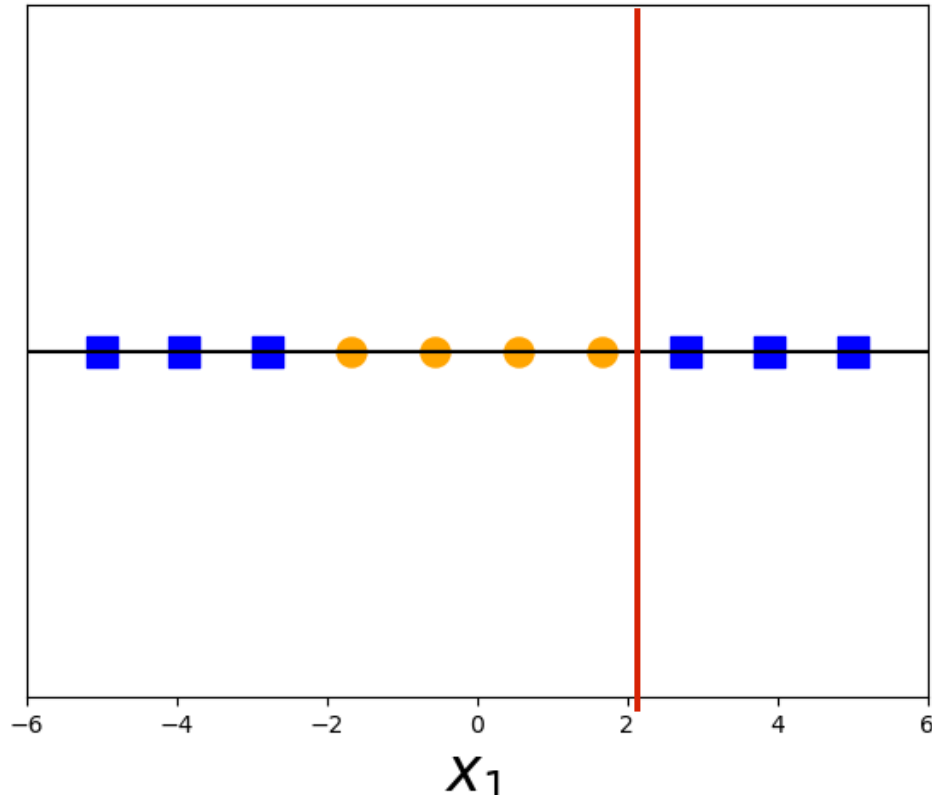
Beispiel Datensatz¹

- Viele echte Datensätze sind nicht linear separierbar
- Zum Beispiel dieser einfache Datensatz mit einem Merkmal $x_i \in X_1 \subset \mathbb{R}$ und den **Klassen**:

$$Y = \{-1, +1\}$$

¹ <https://towardsdatascience.com/the-kernel-trick-c98cdbcaeb3f>

Nichtlinear separierbare Klassen – ein Beispiel

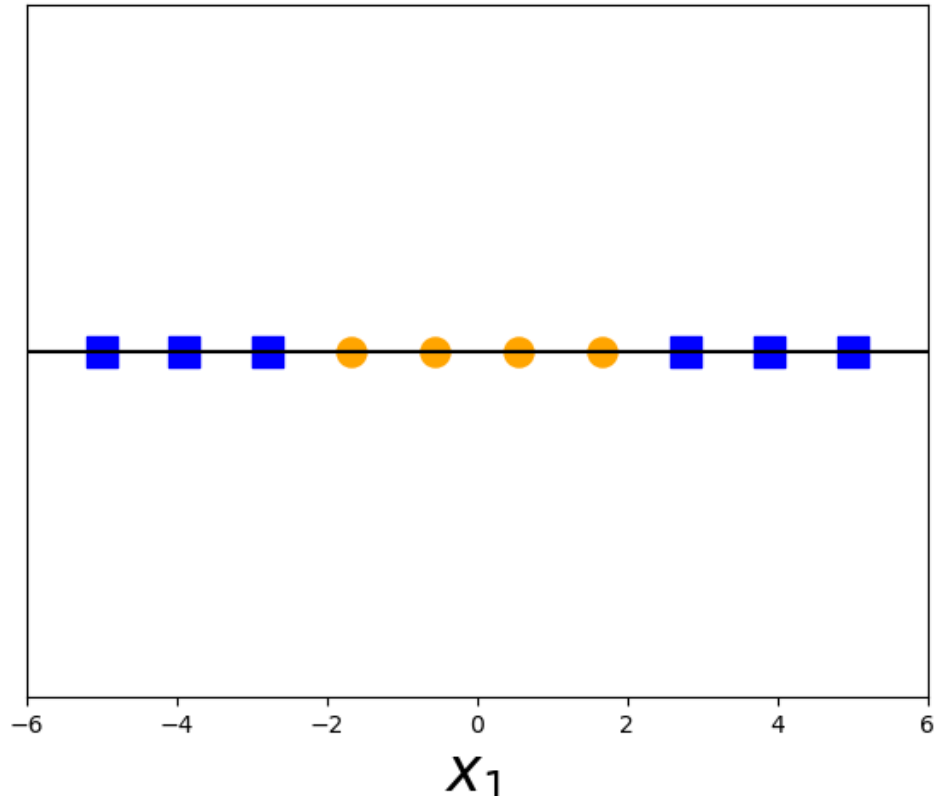


Beispiel Datensatz¹

- Viele echte Datensätze sind nicht linear separierbar
- Zum Beispiel dieser einfache Datensatz mit einem Merkmal $x_i \in X_1 \subset \mathbb{R}$ und den **Klassen**: $Y = \{-1, +1\}$
- Kann nicht linear separiert werden
- Wenn wir trotzdem einen linearen Klassifikator wählen, werden im Trainingsdatensatz immer 30% der Datenpunkte falsch klassifiziert

¹ <https://towardsdatascience.com/the-kernel-trick-c98cdbcaeb3f>

Nichtlinear Separierbare Klassen – ein Beispiel



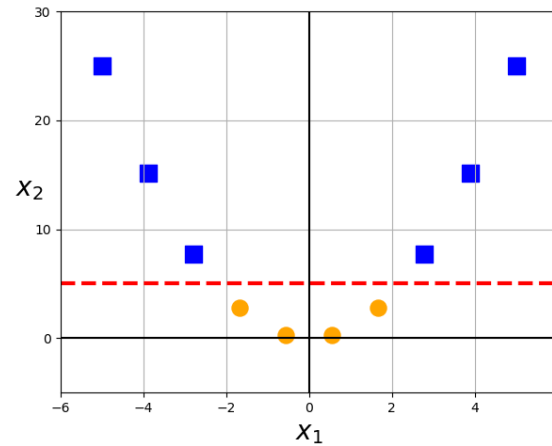
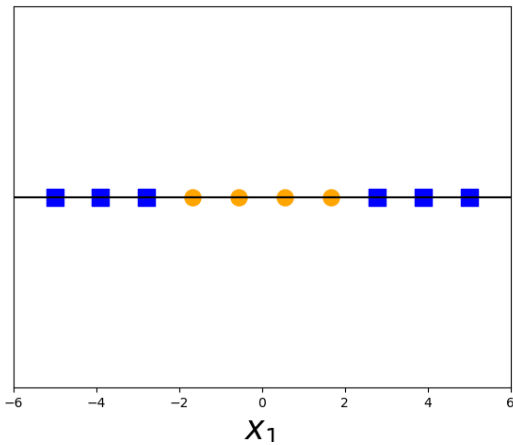
Beispiel Datensatz¹

Neuer Merkmalsraum: ?

In welchem höherdimensionalen Raum ist der Datensatz linear separierbar?

¹ <https://towardsdatascience.com/the-kernel-trick-c98cdbcaeb3f>

Nichtlinear Separierbare Klassen – ein Beispiel



Neuer Merkmalsraum: $\mathbb{R}^2$

In welchem höherdimensionalen Raum ist der Datensatz linear separierbar?

$$\phi: \mathbb{R} \rightarrow \mathbb{R}^2$$

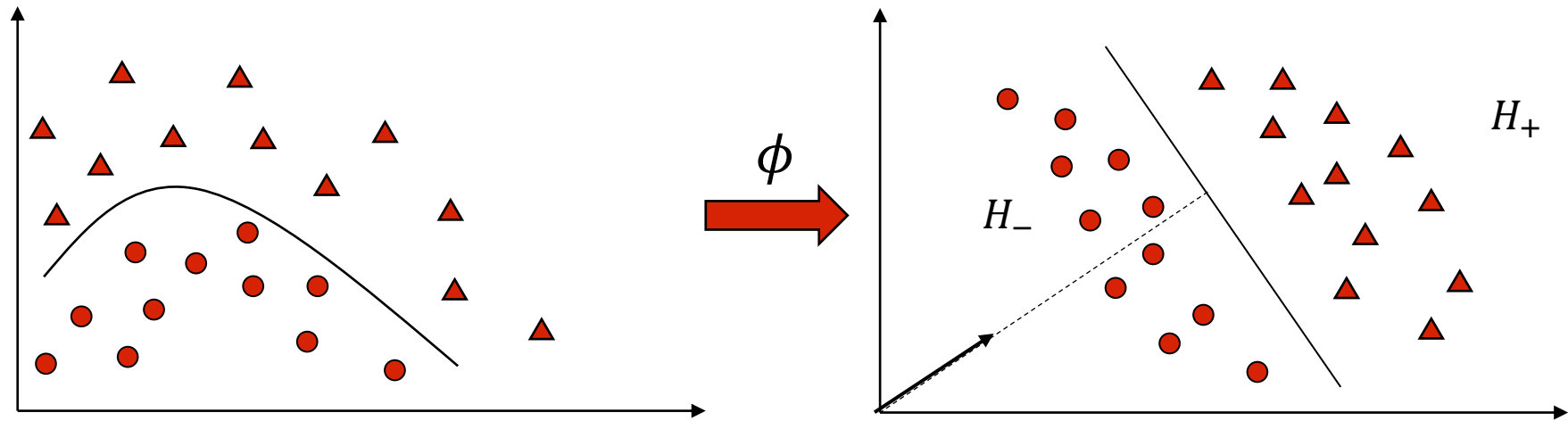
Transformation und linear Klassifikation

¹ <https://towardsdatascience.com/the-kernel-trick-c98cdbcaeb3f>

Kernel Trick – Nichtlineare Klassifikatoren

Was tun, bei nichtlinearen Klassifikationsproblemen?

Idee: Finde eine nichtlineare **Transformation** ϕ , die den Merkmalsraum so abbildet, dass die Klassentrennung im neuen Raum linear erfolgen kann!



Kernel-Trick

Frage: Wie wird die nichtlineare Abbildung ϕ bestimmt?

Antwort: Gar nicht. Zumindest **nicht explizit**.

Lösung: Statt die Transformation ϕ explizit anzugeben und auf die Merkmalsvektoren anzuwenden, **wird das Skalarprodukt** durch eine **nichtlineare Kernelfunktion** k die zwei Vektoren als Input nimmt, ersetzt.

$$\underbrace{\phi(\vec{g})^T \phi(\vec{g}_i)}_{=: k(\vec{g}, \vec{g}_i)}$$

Kernelfunktionen

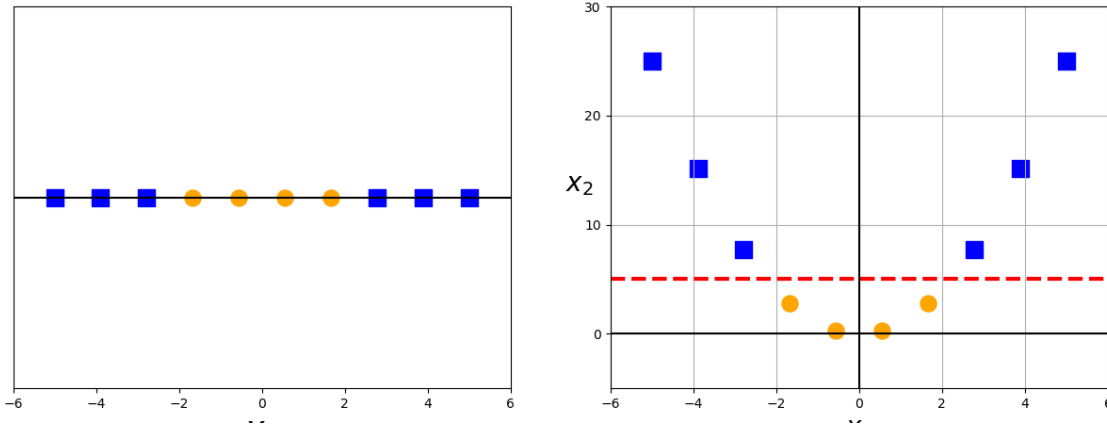
- Kernel-Funktionen sind Funktionen, die zwei Vektoren als Eingabe nehmen und einen Wert ausgeben, der angibt, wie ähnlich sich die Eingabevektoren sind.
- Anstatt explizit in einen höherdimensionalen Raum zu transformieren, verwenden wir den **Kernel-Trick** und **ersetzen das Skalarprodukt im höherdimensionalen Raum durch einen Kernel**.

Gegeben seien Datenpunkte $x_i \in \mathbb{R}^d$ und ein Kern $K(x_i, x_j)$. Der Kernel berechnet das **Skalarprodukt** im transformierten Raum:

$$K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$$

wobei $\phi(x)$ die Abbildung in den höherdimensionalen Raum ist.

Kerneltrick - Beispiel



Transformation und linear Klassifikation

[1 https://towardsdatascience.com/the-kernel-trick-c98cdbcaeb3f](https://towardsdatascience.com/the-kernel-trick-c98cdbcaeb3f)

Statt der Transformation in $\mathbb{R}^2$, hätten wir direkt eine passende **Kernelfunktion** nutzen können

$$\phi: \mathbb{R} \rightarrow \mathbb{R}^2$$

$$K(x, x') = \langle \mathbf{x}, \mathbf{x}' \rangle^d = x^2, \quad d = 2$$

Das ist ein **polynomieller Kernel** mit Dimension 2.

Typische Kernel

Polynomialer Kernel (vom Grad r):

$$k(\vec{u}, \vec{v}) = (\vec{u}^T \vec{v} + c)^r$$

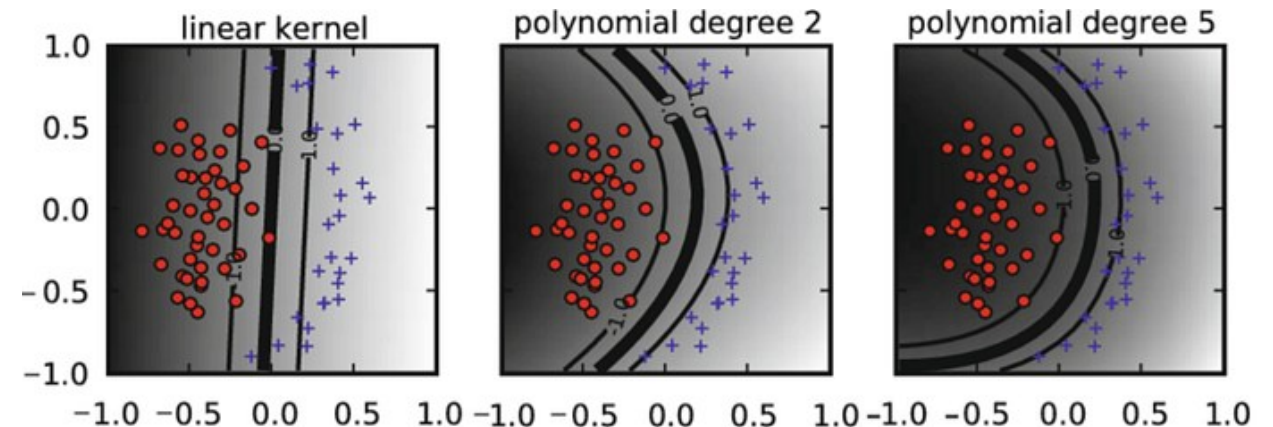
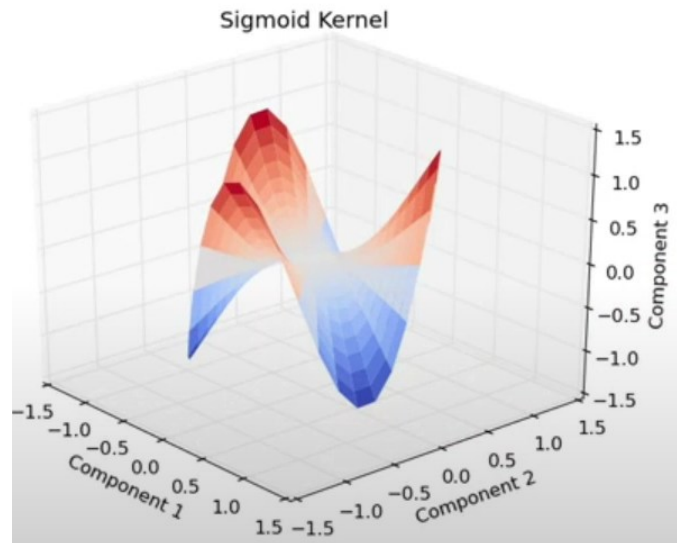
Ein Spezialfall ist der lineare Kernel mit $c = 0$ und $r = 1$.

Dabei handelt es sich gerade um das Skalarprodukt selbst, er bewirkt also keine Änderung

Hyperbolischer Tangenten (Sigmoid) Kernel:

$$k(\vec{u}, \vec{v}) = \tanh(\alpha \vec{u}^T \vec{v} + c)$$

Die Sigmoid-Funktion spielt bei Neuronalen Netzen als Aktivierungsfunktion eine große Rolle



Quelle: <https://medium.com/>

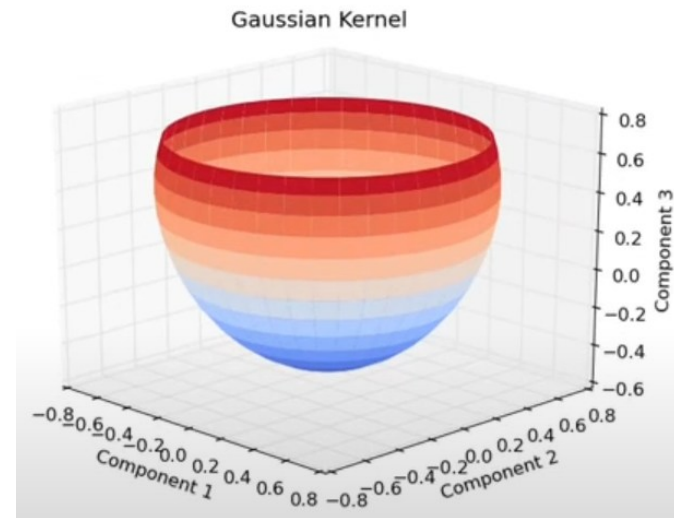
Typische Kernel

Gaußkernel (hier mittelwertfrei mit Kovarianz $\sigma^2 I$):

$$k(\vec{u}, \vec{v}) = e^{-\frac{\|\vec{u}-\vec{v}\|^2}{2\sigma^2}} = e^{-\gamma\|\vec{u}-\vec{v}\|^2} \text{ mit } \gamma = \frac{1}{2\sigma^2}$$

Der Gaußkernel wird auch als **RBF-Kernel** (radial basis function) bezeichnet und entspricht der Dichtefunktion der Normalverteilung $\mathcal{N}(\vec{u} - \vec{v}, \vec{0}, \sigma^2 I)$.

$\mathcal{N}(\vec{x}, \vec{\mu}, \Sigma)$
bezeichnet die
Standard-
Normalverteilung



Quelle: <https://medium.com/>

Vor- und Nachteile des Kernel-Tricks

Vorteile des Kernel-Tricks:

- Die Kernelfunktionen können signifikant effizienter implementiert werden, als durch **explizite Anwendung einer Transformation ϕ** mit anschließendem Skalarprodukt.

Nachteil:

- Die Kernelfunktion muss ausgewählt und parametrisiert werden (Hyperparameter)

In der Regel werden weitere Parameter benötigt

Kernel PCA

Die Idee der **Kernel PCA** ist nun die Kovarianzmatrix (genauer die Gramm-/Skalarproduktmatrix) durch eine **Kernel-Funktion** auf den Daten

$$k(\vec{g}_i, \vec{g}_j) = \langle \phi(\vec{g}_i), \phi(\vec{g}_j) \rangle = \phi(\vec{g}_i)^T \phi(\vec{g}_j)$$

zu **ersetzen** und damit **implizit** eine **nichtlineare Transformation** ϕ auf die Daten anzuwenden.

Man berechnet dazu die **Kernelmatrix**

$$1. \quad K = \left(k(\vec{g}_i, \vec{g}_j) \right)_{ij} \in \mathbb{R}^{N \times N}$$

Und **zentriert** sie (Gram Matrix von K), um die Mittelwertkorrektur durchzuführen. Die Zentrierung der Kernelmatrix erfolgt durch eine lineare Transformation, die sicherstellt, dass der Mittelwert der projizierten Daten im Feature Space null ist.

$$2. \quad \tilde{K} = K - \frac{1}{N} \mathbf{1}_N K - \frac{1}{N} K \mathbf{1}_N + \frac{1}{N^2} K \mathbf{1}_N \mathbf{1}_N^T$$

Die Matrix $\mathbf{1}_N$ ist dabei die (größenangepasste) Matrix, die nur Einsen enthält.

Kernel PCA

Als nächstes berechnet man die **Eigenvektoren und Eigenwerte** der **zentrierten Kernelmatrix** und kann das als **Faktorisierung der Kernelmatrix** darstellen. Die Eigenvektoren sollten normiert werden.

4. $\tilde{K} = U\Sigma U^T$

U enthält die Eigenvektoren (sortiert nach Eigenwert), Σ enthält die Eigenwerte auf der Diagonale

5. Transformation eines x_{neu} :

$$x_{neu} = U^T k_{neu}$$

$$k_{neu} = \begin{bmatrix} K(x_1, x_{neu}) \\ K(x_2, x_{neu}) \\ \vdots \\ K(x_n, x_{neu}) \end{bmatrix}$$

Die **Anzahl der verwendeten Eigenvektoren** definiert dann (wieder) die **Dimension** des Ergebnisraums.

Beispiel

Bezeichne $\vec{x}_1 = (2, -1, -1, 0, 0)$ den (zentrierten) Vektor des ersten Merkmals und $\vec{x}_2 = (4, 0, -2, -2, 0)$ den des zweiten Merkmals (Formulierung analog zur Klausuraufgabe ML_19_SS 3c).

Berechnen Sie für folgende Kernel-Funktionen die Kernel-Matrix.

Linearer Kernel:

$$k(\vec{u}, \vec{v}) = (\vec{u}^T \vec{v} + c)$$

mit $c=1$

Polynomialer Kernel (vom Grad r):

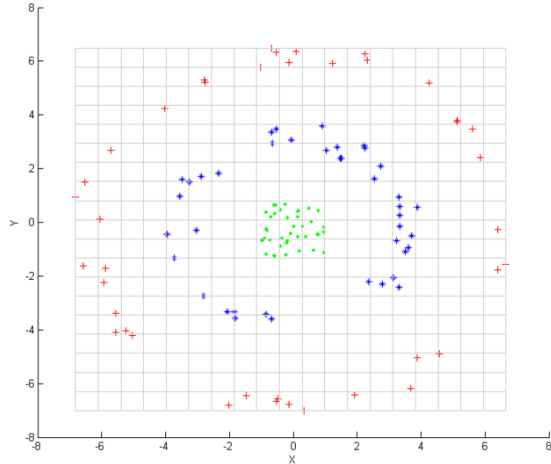
$$k(\vec{u}, \vec{v}) = (\vec{u}^T \vec{v} + c)^r$$

mit $c=1$ und $r=2$

Beispiele Kernel PCA

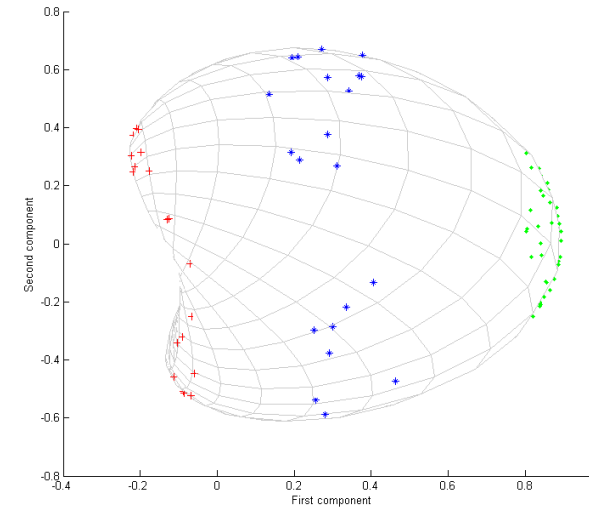
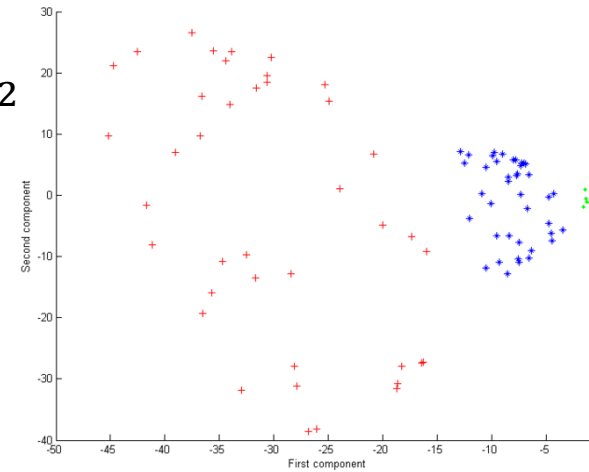
Beispiel:

$$k(\vec{x}_i, \vec{x}_j) = (\vec{x}_i^T \vec{x}_j + 1)^2$$



Originaler
Merkmalsraum

$$k(\vec{x}_i, \vec{x}_j) = e^{-\frac{\|\vec{x}_i - \vec{x}_j\|^2}{2\sigma^2}}$$



Merkmale nach Anwendung
der PCA

Bildquelle Wikipedia

Übung

1. Berechne die Kernel Matrix für einen lineare Kernel mit folgenden Daten:

Datenpunkte

Wir haben 3 Datenpunkte in 2D:

$$x_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad x_2 = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \quad x_3 = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

Lineare Kernel-Matrix

Der lineare Kernel ist definiert als:

$$K(x_i, x_j) = x_i^T x_j$$

Die Kernel-Matrix K ist:

$$K = \begin{bmatrix} x_1^T x_1 & x_1^T x_2 & x_1^T x_3 \\ x_2^T x_1 & x_2^T x_2 & x_2^T x_3 \\ x_3^T x_1 & x_3^T x_2 & x_3^T x_3 \end{bmatrix}$$

2. Zentriere die Kernelmatrix

Die zentrierte Kernel-Matrix $\tilde{K}$ wird berechnet als:

$$\tilde{K} = K - \frac{1}{n} \mathbf{1}_n K - \frac{1}{n} K \mathbf{1}_n + \frac{1}{n^2} \mathbf{1}_n K \mathbf{1}_n$$

mit $n = 3$ und $\mathbf{1}_n$ als 3×3 -Matrix mit Einsen.

Vorteile und Nachteile Kernel PCA

Vorteile gegenüber der normalen PCA

- Kernel PCA kann nichtlineare Strukturen in den Daten erfassen, indem es die Daten in einen höherdimensionalen Raum projiziert. Dies ist besonders nützlich, wenn die Daten nichtlinear verteilt sind.
- Durch die Verwendung verschiedener Kernel-Funktionen (z. B. polynomial, radial basis function) kann Kernel PCA an die spezifischen Eigenschaften der Daten angepasst werden, was die Analyse verbessert.
- Kernel PCA kann wichtige Merkmale der Daten bewahren, die in einer linearen Analyse möglicherweise verloren gehen würden. Dies führt zu einer besseren Repräsentation der Daten.

Nachteile:

- Abhängigkeit von Wahl des Kernels und dessen Parameter
- Overfitting wenn zu komplexer Kernel
- Empfindlich gegenüber von Skalierung

Links

- [Bishop-Pattern-Recognition-and-Machine-Learning](#) Kapitel 7.1 (SVM), Kapitel 6 Intro und 6.1 (Kernel Methods)
- [Mathematics for Machine Learning](#) - SVM Kapitel 12 - S.317-347
- [Kernel PCA Tutorial](#)

